

VIETNAM NATIONAL UNIVERSITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



REPORT
SPECIALIZED PROJECT (CO4029)

INTELLIGENT CODE ASSISTANCE FOR WEB DEVELOPMENT: RAG-ENHANCED LLM APPROACH

Major: Computer Science

Supervisors: Nguyen An Khuong, PhD
Pham Nhut Huy
Students: Nguyen Trang Sy Lam – Student ID: 2152715
Bui Ho Hai Dang – Student ID: 2153289
Le Hoang Phuc – Student ID: 2152239

Ho Chi Minh City, December 2024

Declaration

We hereby declare that this research is entirely our own work, conducted under the supervision and guidance of Dr. Nguyen An Khuong and Mr. Pham Nhut Huy. The results presented in this research are legitimate and have not been published in any form prior to this. All materials used in this research have been collected by us from various sources and are properly cited in the bibliography section. Additionally, all figures included in this report have been created by us.

This research also references results from other authors, which have already been published by the respective authors.

In the event of any plagiarism, we take full responsibility for our actions. Ho Chi Minh City University of Technology - Vietnam National University is not liable for any copyright infringements arising from this research.

Ho Chi Minh City, December 2024

Authors

Acknowledgements

Firstly, we would like to sincerely express our gratitude to our supervisors, Dr. Nguyen An Khuong, and Mr. Pham Nhut Huy. To Dr. Nguyen An Khuong, thank you for dedicating your precious time to us. The weekly meetings helped our team stay on track, and your timely feedback, advice, and guidance greatly contributed to the success of our research. To Mr. Pham Nhut Huy, your expertise in the field, along with your thesis writing and presentation skills, was invaluable to our progress.

Secondly, we wish to extend our heartfelt thanks to our families for their unwavering support, both emotionally and financially, throughout this research process. Our accomplishments would not have been possible without their encouragement and assistance.

Lastly, this is from Nguyen Trang Sy Lam, the leader of the team. I would like to personally thank Dr. Nguyen An Khuong for guiding me through the most challenging days of my student life. When I was in a dark place, your support and mentorship pulled me through and helped me complete this research. Words cannot fully convey my gratitude to you.

Abstract

In this thesis, we present the design of a Retrieval-Augmented Generation (RAG) pipeline integrated into a Visual Studio Code (VSCode) extension, serving as an intelligent assistant that run locally for web developers working with the Next.js framework. The pipeline leverages Milvus [50] as the vector database to efficiently store and retrieve information from Next.js documentation, prepared during the data processing phase. For response generation, the system utilizes Alibaba's Qwen [3], a Large Language Model (LLM). This combination of technologies ensures the system providing accurate, context-aware assistance. By retrieving relevant information and generating responses grounded in the retrieved data, the proposed design enhances productivity and simplifies the development process within the VSCode environment.

Contents

Declaration	i
Acknowledgements	ii
Abstract	iii
1 Introduction	1
1.1 Motivations	1
1.2 Objectives	2
1.3 Scope	2
1.4 Challenges	3
1.5 Structure of the report	3
2 Preliminaries	4
2.1 Retriever	4
2.1.1 Preprocessing	5
2.1.2 Embedding	5
2.1.3 Hierarchical Navigable Small World (HNSW) Indexing	6
2.1.4 Pre-retrieval	10
2.1.5 Retrieval	11
2.1.6 Post-Retrieval	12
2.2 Generator	13
2.2.1 Transformer	13
2.2.2 Prompt engineering	14
2.3 Augmentation methods	15
2.3.1 Augmentation stage: Inference	15
2.3.2 Augmentation source: Structured data	17
2.3.3 Augmentation process	17
3 Related works and tools	20
3.1 RAG and LLMs in Coding Tasks	20
3.1.1 Retrieval Strategies	20
3.1.2 Generative Enhancements	22
3.1.3 Specialized Applications	24
3.2 Developer Assistance Tools	24
3.2.1 Functionality	24
3.2.2 Retrieval Mechanisms	26
3.2.3 Code Generation Capabilities	26
3.2.4 Integration in Development Processes	26

4	Approach	28
4.1	Fine-tuning vs. RAG	28
4.1.1	Fine-tuning	28
4.1.2	Comparison between fine-tuning and RAG	29
4.1.3	Rationale for Choosing RAG	29
4.2	Why BGE-M3?	30
4.3	Why Qwen?	30
4.4	Main architecture	31
4.4.1	Local Vector Storage	31
4.4.2	VSCode integration	34
4.4.3	Feedback loop	34
5	Initial Implementation	35
5.1	Data Structure Design	35
5.1.1	Data structure design	36
5.1.2	Fields in the Schema	37
5.2	Data Preparation	38
5.3	Data Preprocessing	38
5.4	Database Setup	39
5.4.1	Deploying Milvus with Docker Compose	39
5.4.2	Installing PyMilvus	40
5.4.3	Creating Indices	40
5.4.4	Inserting Data into the Collection	40
5.4.5	Hybrid search implementation	40
5.5	Local LLMs setup	40
5.5.1	Setup Ollama	41
5.5.2	Integration with the RAG System	41
6	Evaluation plan	42
6.1	Evaluation metrics	42
6.1.1	Retriever metrics:	42
6.1.2	Generator metrics:	42
6.1.3	System metrics:	43
6.2	Evaluation methodology	43
6.2.1	Retriever evaluation:	43
6.2.2	Generator evaluation:	43
6.2.3	System evaluation:	43
7	Future plan	44
7.1	Overview	44
7.2	Current tasks: late December 2024	44
7.3	Upcoming tasks: January 2025 and Beyond	44
	Bibliography	46

1.1 Motivations

The process of web development has been significantly transformed by the introduction and adoption of frameworks. Frameworks have become the industry standard, providing developers with structured, reusable tools that simplify the creation of web applications. This evolution has also been greatly influenced by the rise of LLMs like OpenAI's Codex [9], Google's Gemini [46], and GitHub Copilot¹, which have made coding more accessible by offering automated code generation, real-time suggestions, and debugging assistance. These tools have made entry into web development easier than ever, lowering barriers for beginners and speeding up development processes.

However, despite these advancements, LLMs still face certain limitations, such as hallucinations, difficulties in referencing external sources, and performance issues when handling specific tasks. A promising solution to mitigate these limitations is using RAG. RAG enhances the generative capabilities of LLMs by retrieving relevant external information before generating content, improving both the accuracy and relevance of the output. This technique is particularly beneficial in domains like web development, where staying up-to-date with framework versions and ensuring compatibility across services is critical. Incorporating RAG helps bridge the gap between rapidly evolving frameworks and the knowledge developers need to make informed decisions about the tools and technologies they use.

¹Github Copilot - <https://copilot.github.com/>

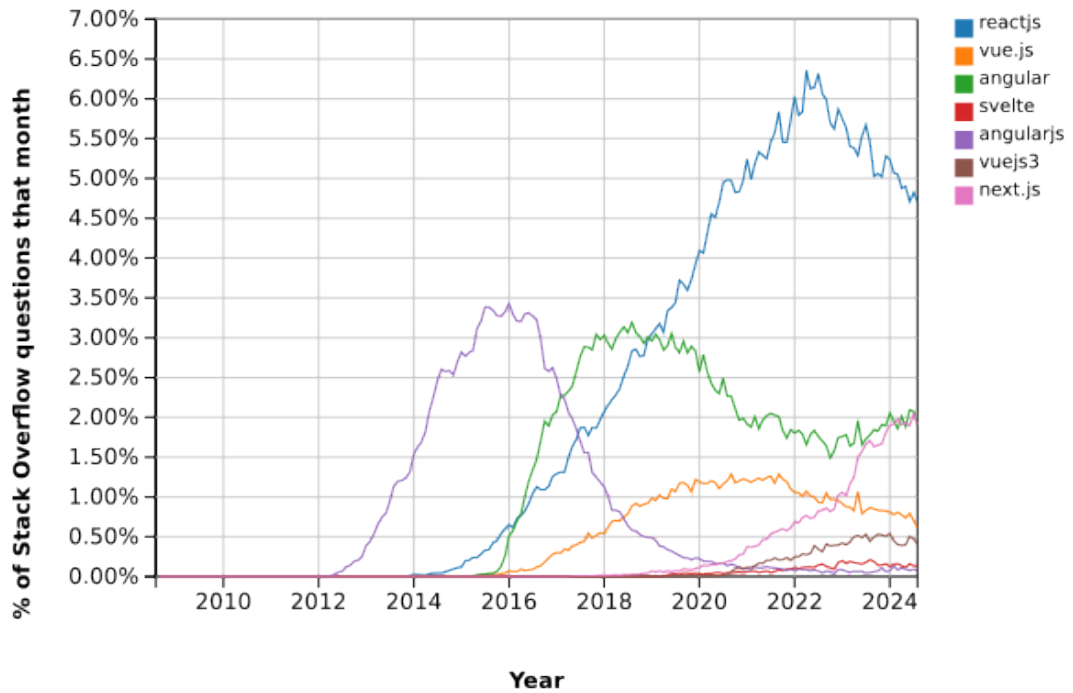


Figure 1.1: The popularity of frameworks based on StackOverflow questions 2008-2024

To illustrate, consider the Next.js framework, which has gained significant popularity in recent years. Built on top of React.js (currently the most popular front-end library), Next.js has seen rapid growth and frequent updates. For instance, version 13.0.0 of Next.js was released on October 25th, 2022, followed by multiple version 13.x.x updates, until version 14.0.0 was released on October 26th, 2023. Just 1 year later, they released version 15.0.0 for the framework with many breaking changes². This rapid pace of development introduces challenges for developers who need to maintain or upgrade their codebases, especially considering that React.js itself has also undergone significant changes, such as the switch from version 18.0.0 to 19.0.0, which involved major updates³.

1.2 Objectives

The objective of this research is to develop a tool, specifically a VSCode extension, that facilitates the web development process by addressing common challenges associated with modern frameworks. The tool aims to:

- Utilize RAG to assist developers with question answering, code generation, and code review. RAG will retrieve relevant information from documentation or community forums to enhance the accuracy and relevance of the tool's suggestions.
- Analyze the user's codebase and suggest modifications for parts that are no longer relevant, particularly when updating the code to newer versions of a framework.

This extension will help streamline the development process, making it easier for developers to maintain, upgrade, and enhance their applications with minimal friction.

1.3 Scope

The scope is intentionally confined to Next.js as the primary framework for web application development, enabling a focused exploration of its interaction with RAG methodologies. Additionally, the implementation targets applications written in JavaScript, TypeScript, HTML, and CSS, ensuring compatibility with commonly used web development programming languages.

²The latest Next.js news - <https://nextjs.org/blog>

³How to Upgrade to React 18 - <https://react.dev/blog/2024/04/25/react-19-upgrade-guide>

1.4 Challenges

Developing this tool involves several challenges:

- **Efficient Context Management:** Since the goal is to make the tool lightweight, storing and managing the context of a user's codebase in a way that allows for fast retrieval without overwhelming system resources is a major challenge.
- **Data Curation Complexity:** The data curation process requires extensive knowledge about the framework.
- **Handling Framework Updates:** Keeping the tool updated with the frequent changes in frameworks like Next.js and their associated packages is challenging. Continuous updates are needed to ensure compatibility and provide accurate suggestions.
- **User Experience:** Balancing the amount of information presented to the user without overwhelming them is essential. The tool should provide meaningful insights and recommendations without causing information overload.

1.5 Structure of the report

Chapter 1. Introduction. This chapter introduces the motivations, objectives, and scope of the research. It highlights the challenges of web development and the role of RAG in enhancing efficiency and accuracy. The chapter outlines the structure of the report, providing a roadmap for the research presented.

Chapter 2. Preliminaries. This chapter provides foundational knowledge on RAG, emphasizing its retriever and generator components. It discusses indexing, vector embeddings, and augmentation methods, offering a clear understanding of the technologies used. Key concepts of the Next.js framework and associated development tools are also introduced.

Chapter 3. Related Works and Tools. This chapter explores prior research on RAG and its application in web development. It reviews developer tools such as VSCode extensions and discusses strategies for retrieval and augmentation. Comparative analyses of existing frameworks and methodologies are also presented.

Chapter 4. Approach. This chapter describes the proposed solution, including the architecture of the RAG-enhanced system. It explains the rationale for selecting specific models like BGE-M3 and Qwen and details the integration of local vector storage with the Next.js framework to ensure efficient retrieval and generation.

Chapter 5. Initial implementation. This chapter outlines the step-by-step development process of the RAG pipeline. It includes data preparation, database setup, and integration with the VSCode extension. Challenges encountered during implementation and their solutions are also discussed.

Chapter 6. Evaluation plan. This chapter evaluates the system using metrics such as accuracy and developer productivity. It describes the testing methodology, compares results with existing solutions, and highlights the strengths and limitations of the proposed approach.

Chapter 7. Future plan. The final chapter summarizes the contributions of the research and discusses potential areas for improvement. It offers recommendations for extending the system's compatibility and incorporating advanced retrieval techniques to enhance performance and scalability.

The RAG process combines the capabilities of retrieval systems and generative AI to deliver precise and context-aware responses. It begins with query vectorization, where the user's input is transformed into a mathematical vector using models like BERT or sentence transformers. This vector is then used to search a pre-indexed vector database for semantically similar entries, retrieving the most relevant data. Once retrieved, the data undergoes preprocessing to ensure compatibility with the language model, such as summarizing or structuring it. The refined data is then integrated into the input context of a LLM, enabling it to generate responses that blend the retrieved information with its broader pre-trained knowledge. This structured pipeline ensures that the generated outputs are accurate, specific, and contextually grounded.

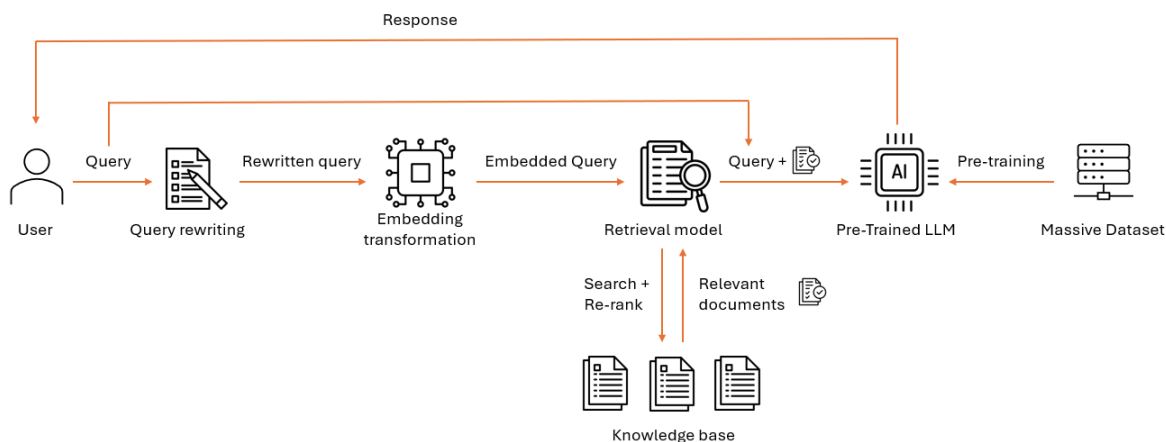


Figure 2.1: Retrieval-Augmented Generation Process

The RAG process is composed of two main components: the retriever and the generator. The retriever is responsible for locating and extracting relevant information from an external knowledge source, employing techniques such as keyword-based search, semantic similarity search, or neural network-based retrieval to ensure precise results. The generator then utilizes the retrieved context to produce coherent and accurate responses, ensuring the output aligns with the user's query while maintaining relevance and context. Together, these components form a robust framework for leveraging external knowledge in generative tasks.

2.1 Retriever

The retriever stage involves finding and gathering relevant text passages from an external knowledge source. This can be done through various methods such as keyword-based search, semantic similarity

search, or neural network-based retrieval techniques. One major application of embedding models is neural retrieval. By measuring the semantic relationship with the text embeddings, allowing for the comparison of text based on content rather than just keyword matching, then the relevant answers to the input query can be retrieved based on the embedding similarity.

2.1.1 Preprocessing

Preprocessing starts with the cleaning and extraction of raw data in diverse formats like PDF, HTML, Word, and Markdown, which is then converted into a uniform plain text format. To accommodate the context limitations of language models, text is segmented into smaller, digestible chunks. Chunks are then encoded into vector representations using an embedding model and stored in a vector database. This step is crucial for enabling efficient similarity searches in the subsequent retrieval phase [14].

There are ways to optimize the indexing process to enhance the performance of the upcoming retrieval steps:

1. **Chunking strategy:** Documents can be splitted into chunks using various methods like Fixed-Size Chunking (for structured documents and surveys), Recursive Chunking (for documents with large amounts of data), Sentence-Aware Chunking (for analysis of textbooks) and Semantic Chunking (for Information Retrieval systems and legal document analysis) [23].
2. **Metadata attachments:** Each chunk can include extra information (page number, file name, author, category, or timestamp) as metadata. Afterward, the retrieval process can be refined by filtering results based on metadata, which helps narrow down the search. Assigning different weights to document timestamps during retrieval can achieve time-aware RAG, ensuring the freshness of knowledge and avoiding outdated information [14].

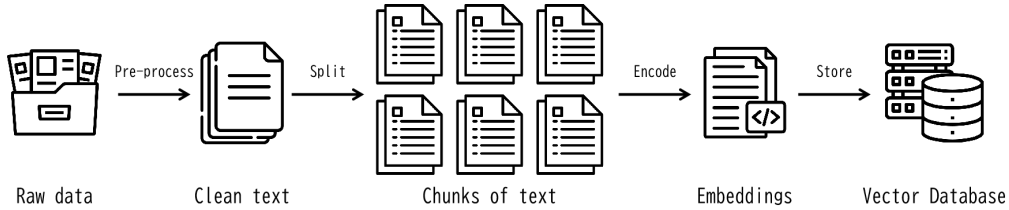


Figure 2.2: Preprocessing process for retriever

2.1.2 Embedding

Embeddings are a key component of RAG systems, which are used to effectively retrieve pertinent information from big datasets. By converting textual data into numerical representations, these embeddings are vector-based representations, typically generated by neural networks, that transform input text into a structured vector space. In this space, proximity reflects the semantic relationships of the input.

In essence, an embedding is a vector, a high-dimensional representation of a text in which every dimension represents a distinct facet of meaning. Consider a two-dimensional Cartesian coordinate system from the algebra class, but with a lot more dimensions (usually 768, 1024, or 1536) to better visualize this. Models such as BERT (Bidirectional Encoder Representations from Transformers) [11] and Sentence-BERT [38] employ this high-dimensional vector space to represent the semantic features of text. By calculating the distance between these vectors, embeddings enable the system to extract pertinent information from large datasets and choose the most contextually relevant text passages for response generation. This feature is essential for enhancing large language models' (LLMs') outputs' contextual relevance and accuracy.

Many embedding techniques have been used over the years:

1. **Word2Vec** [30]: A well-liked technique that uses the Continuous Bag of Words (CBOW) or Skip-gram models to learn word embeddings depending on local context. Word2Vec is effective at many jobs, but it has a major drawback: it has trouble handling polysemy, which is the situation where words have several meanings depending on the context. This is because Word2Vec creates a single representation for every word, independent of how it is used.
2. **BERT** [11]: A transformer-based model that generates contextual embeddings, considering both the preceding and following words in a sentence. This allows BERT to produce more accurate

representations of words based on their surrounding context. It has been widely adopted for many NLP tasks such as question answering, sentiment analysis, and named entity recognition.

3. **Sentence-BERT (SBERT)** [38]: An extension of BERT, fine-tuned specifically for generating sentence embeddings. Unlike traditional BERT, which produces word-level embeddings, SBERT produces fixed-size embeddings for entire sentences. This allows for efficient comparison of sentence pairs, making SBERT highly effective for tasks like semantic textual similarity, paraphrase identification, and information retrieval.
4. **FAISS [12] and Dense Retrieval Techniques:** In RAG systems, embeddings are often used with specialized retrieval tools like FAISS (Facebook AI Similarity Search) to perform fast, approximate nearest neighbor search. This helps locate the most relevant pieces of information from a large corpus, making the retrieval process both scalable and efficient.

Moreover, our embedding can be trained in different ways:

1. **Supervised Learning:** In supervised learning, embeddings are trained with labeled data to predict specific outcomes, such as class labels or relationships. Examples include training embeddings for tasks like sentiment analysis or named entity recognition (NER).
2. **Unsupervised Learning:** Unsupervised learning allows embeddings to be trained without labels, often by predicting words or context from surrounding text. Models like Word2Vec and GloVe use this method to learn semantic relationships between words from large corpora.

Fine-tune Embedding models: Reinforcement Learning [24]: In reinforcement learning, embeddings are dynamically adjusted based on feedback from the environment to improve the system’s performance over time. For instance, in a Retrieval-Augmented Generation (RAG) system designed for domain-specific chatbots, reinforcement learning is employed to optimize the retrieval of relevant context. Using Reinforcement Learning with Human Feedback (RLHF) [24], models like GPT-4 are fine-tuned to respond accurately to domain-specific queries. A key example is optimizing FAQ retrieval for customer service chatbots. In such systems, reinforcement learning is used to decide whether to fetch additional context for each user query or rely on previously retrieved information. The system evaluates the quality of the generated responses (e.g., by GPT-4) and uses this evaluation as a reward signal. For example, if the system correctly answers a follow-up query without fetching redundant FAQ context, it receives a positive reward, thereby learning to minimize unnecessary retrievals. This approach not only improves efficiency by saving computational resources (such as API token usage) but also enhances accuracy by reducing noise in the generated outputs.

2.1.3 Hierarchical Navigable Small World (HNSW) Indexing

1. Vector database and HNSW:

In RAG systems, vector database plays a pivotal role in efficiently retrieving relevant information to augment the capabilities of generative language models. These databases are designed to store and query high-dimensional embeddings, which are numerical representations of text, code, images, or other data. Embeddings capture semantic relationships between data points, enabling similarity searches based on meaning rather than exact matches. This makes vector databases a cornerstone of modern RAG pipelines, ensuring that retrieved context aligns with the query’s intent. To achieve efficient and scalable retrieval from large embedding datasets, RAG systems often employ Approximate Nearest Neighbor (ANN) algorithms, with HNSW graphs [29] being one of the most widely used techniques.

2. HNSW algorithm:

The HNSW algorithm is built on the concept of organizing links based on their length scales across multiple layers, enabling efficient search within a hierarchical, multilayer graph structure.

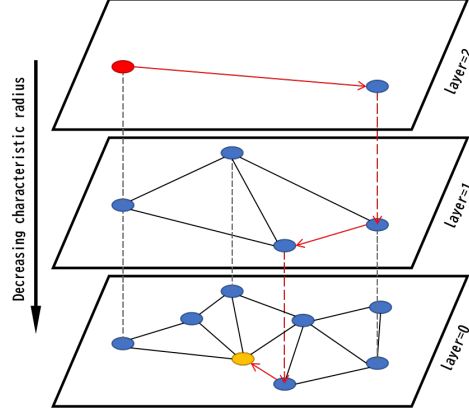


Figure 2.3: HNSW search algorithm visualization. The search starts from entry point from top layer (shown red), red arrows show direction of greedy algorithm to query point (shown yellow) [29].

The HNSW algorithm begins with a network construction phase (Algorithm 1), where the graph structure is incrementally built through the consecutive insertion of stored elements. For each new element, a maximum layer l is randomly determined using an exponentially decaying probability distribution. This hierarchical organization ensures that higher layers contain progressively fewer nodes, optimizing the search process by focusing on coarse-to-fine navigation.

The insertion process begins at the top layer of the hierarchical graph, where the algorithm performs a greedy traversal to locate the ef closest neighbors to the newly inserted element q within that layer. Once these nearest neighbors are identified, they serve as entry points for the next layer. This process is repeated layer by layer, moving downward through the graph, ensuring that the inserted element is effectively connected to its most relevant neighbors at each level.

Algorithm 1 INSERT(**hsnw**, q , M , $M_{\max}$, **efConstruction**, mL)

Input: multilayer graph **hsnw**, new element q , number of established connections M , maximum number of connections per element per layer $M_{\max}$, size of the dynamic candidate list **efConstruction**, normalization factor for level generation mL

Output: update **hsnw** inserting element q

```
1:  $W \leftarrow \emptyset$  ▷ list for the currently found nearest elements
2:  $ep \leftarrow$  get enter point for hsnw
3:  $L \leftarrow$  level of  $ep$  ▷ top layer for hsnw
4:  $l \leftarrow -\lfloor \ln(\text{unif}(0, 1)) \cdot mL \rfloor$  ▷ new element's level
5:  $l \leftarrow \min(l, L + 1)$ 
6: for  $lc \leftarrow L$  to  $l + 1$  do
7:    $W \leftarrow \text{SEARCH-LAYER}(q, ep, \text{ef} = 1, lc)$ 
8:    $ep \leftarrow$  get the nearest element from  $W$  to  $q$ 
9: end for
10: for  $lc \leftarrow \min(L, l)$  to 0 do
11:    $W \leftarrow \text{SEARCH-LAYER}(q, ep, \text{efConstruction}, lc)$ 
12:   neighbors  $\leftarrow \text{SELECT-NEIGHBORS}(q, W, M, lc)$  ▷ Algorithm 3 or 4
13:   add bidirectional connections from neighbors to  $q$  at layer  $lc$ 
14:   for each  $e \in \text{neighbors}$  do
15:      $eComm \leftarrow \text{neighborhood}(e)$  at layer  $lc$ 
16:     if  $|eComm| > M_{\max}$  then ▷ shrink connections if needed
17:       if  $lc = 0$  then ▷ if  $lc = 0$  then  $M_{\max} = M_{\max 0}$ 
18:          $M_{\max} \leftarrow M_{\max 0}$ 
19:       end if
20:        $eComm \leftarrow \text{SELECT-NEIGHBORS}(e, eComm, M_{\max}, lc)$  ▷ Algorithm 3 or 4
21:     end if
22:     set neighborhood( $e$ ) at layer  $lc$  to  $eComm$ 
23:   end for
24: end for
25:  $ep \leftarrow W$ 
26: set enter point for hsnw to  $q$ 
```

Closest neighbors at each layer are identified using a variant of the greedy search algorithm (as outlined in Algorithm 2). To approximate the *ef* nearest neighbors at a given layer l_c , the algorithm maintains a dynamic list W , which stores the *ef* closest found elements. This list is initially populated with the entry points and is iteratively updated during the search process.

Algorithm 2 SEARCH-LAYER(q, ep, ef, lc)

Input: Query element q , enter points ep , number of nearest to q elements to return ef , layer number lc

Output: ef closest neighbors to q

```
1:  $v \leftarrow ep$  ▷ Set of visited elements
2:  $C \leftarrow ep$  ▷ Set of candidates
3:  $W \leftarrow ep$  ▷ Dynamic list of found nearest neighbors
4: while  $|C| > 0$  do
5:    $c \leftarrow$  Extract nearest element from  $C$  to  $q$ 
6:    $f \leftarrow$  Get furthest element from  $W$  to  $q$ 
7:   if  $\text{distance}(c, q) > \text{distance}(f, q)$  then
8:     break ▷ All elements in  $W$  are evaluated
9:   end if
10:  for each  $e \in \text{neighbourhood}(c)$  at layer  $lc$  do ▷ Update  $C$  and  $W$ 
11:    if  $e \notin v$  then
12:       $v \leftarrow v \cup e$ 
13:       $f \leftarrow$  Get furthest element from  $W$  to  $q$ 
14:      if  $\text{distance}(e, q) < \text{distance}(f, q)$  or  $|W| < ef$  then
15:         $C \leftarrow C \cup e$ 
16:         $W \leftarrow W \cup e$ 
17:        if  $|W| > ef$  then
18:          Remove furthest element from  $W$  to  $q$ 
19:        end if
20:      end if
21:    end if
22:  end for
23: end while
24: return  $W$ 
```

At each step, the algorithm evaluates the neighborhood of the closest, yet-to-be-evaluated element in W . The search continues until the neighborhoods of all elements in W have been evaluated. Unlike traditional approaches that limit the number of distance calculations, the HNSW stop condition offers a key advantage: it discards candidates for evaluation that are farther from the query than the furthest element already in W . This approach prevents unnecessary evaluations and avoids bloating the search structures, ensuring a more efficient and streamlined search process.

In the HNSW algorithm, two methods are employed to select MM neighbors from a pool of candidates during the graph construction phase: Simple (Algorithm 3) and Heuristic (Algorithm 4).

Algorithm 3 SELECT-NEIGHBORS-SIMPLE(q, C, M)

Input: Base element q , candidate elements C , number of neighbors to return M

Output: M nearest elements to q

```
1: return  $M$  nearest elements from  $C$  to  $q$ 
```

Algorithm 4 SELECT-NEIGHBORS-HEURISTIC($q, C, M, lc, \text{extendCandidates}, \text{keepPrunedConnections}$)

Input: Base element q , candidate elements C , number of neighbors to return M , layer number lc , flag indicating whether or not to extend candidate list **extendCandidates**, flag indicating whether or not to add discarded elements **keepPrunedConnections**

Output: M elements selected by the heuristic

```

1:  $R \leftarrow \emptyset$ 
2:  $W \leftarrow C$ 
3: if extendCandidates then
4:   for each  $e \in C$  do
5:     for each  $e_{\text{adj}} \in \text{neighbourhood}(e)$  at layer  $lc$  do
6:       if  $e_{\text{adj}} \notin W$  then
7:          $W \leftarrow W \cup e_{\text{adj}}$ 
8:       end if
9:     end for
10:  end for
11: end if
12:  $W_d \leftarrow \emptyset$ 
13: while  $|W| > 0$  and  $|R| < M$  do
14:    $e \leftarrow$  Extract nearest element from  $W$  to  $q$ 
15:   if  $e$  is closer to  $q$  compared to any element from  $R$  then
16:      $R \leftarrow R \cup e$ 
17:   else
18:      $W_d \leftarrow W_d \cup e$ 
19:   end if
20: end while
21: if keepPrunedConnections then
22:   while  $|W_d| > 0$  and  $|R| < M$  do
23:      $R \leftarrow R \cup$  Extract nearest element from  $W_d$  to  $q$ 
24:   end while
25: end if
26: return  $R$ 

```

▷ Working queue for the candidates
▷ Extend candidates by their neighbors
▷ Queue for the discarded candidates
▷ Add some of the discarded connections from W_d

The K-Approximate Nearest Neighbor Search (K-ANNS) algorithm, as implemented in the HNSW framework, is detailed in Algorithm 5. It is conceptually similar to the insertion process of an item with a designated top layer $l = 0$. However, instead of establishing new connections, the goal of the K-ANNS algorithm is to identify and return the closest neighbors in the graph's ground layer as the final search results.

Algorithm 5 K-ANN-SEARCH(hsnw, q, K, ef)

Input: Multilayer graph **hsnw**, query element q , number of nearest neighbors to return K , size of the dynamic candidate list ef

Output: K nearest elements to q

```

1:  $W \leftarrow \emptyset$ 
2:  $ep \leftarrow$  Get enter point for hsnw
3:  $L \leftarrow$  Level of  $ep$ 
4: for  $lc \leftarrow L \dots 1$  do
5:    $W \leftarrow \text{SEARCH-LAYER}(q, ep, ef = 1, lc)$ 
6:    $ep \leftarrow$  Get nearest element from  $W$  to  $q$ 
7: end for
8:  $W \leftarrow \text{SEARCH-LAYER}(q, ep, ef, lc = 0)$ 
9: return  $K$  nearest elements from  $W$  to  $q$ 

```

▷ Set for the current nearest elements
▷ Top layer for **hsnw**

2.1.4 Pre-retrieval

Query expansion: In the context of RAG, a single query can be transformed and expanded into multiple variations that are semantically enriched, capturing different nuances and interpretations of the original

input. By diversifying the queries, the system increases the likelihood of retrieving contextually specific and relevant information from the knowledge base. This enriched retrieval process ensures that the generated responses are not only more precise but also better aligned with the user's intent, ultimately enhancing the quality and relevance of the output provided by the model [14].

Query rewriting: The original queries are not always optimal for LLM retrieval, especially in real-world scenarios. Therefore, we can prompt LLM to rewrite the queries [14]. Query rewriting in RAG refers to the process of refining or reformulating user queries to enhance the retrieval module's effectiveness. This step is critical in ensuring that the retriever fetches highly relevant and accurate documents or context from the knowledge base. Effective query rewriting includes disambiguating vague queries, expanding them with synonyms or related terms, correcting errors, and incorporating historical or contextual information. By improving the quality of the input to the retriever, query rewriting significantly boosts the overall performance of the RAG system, leading to more accurate and contextually appropriate outputs from the generation module. A user query can be transformed and decomposed in many ways before being executed as part of a RAG query engine, agent, or any other pipeline.

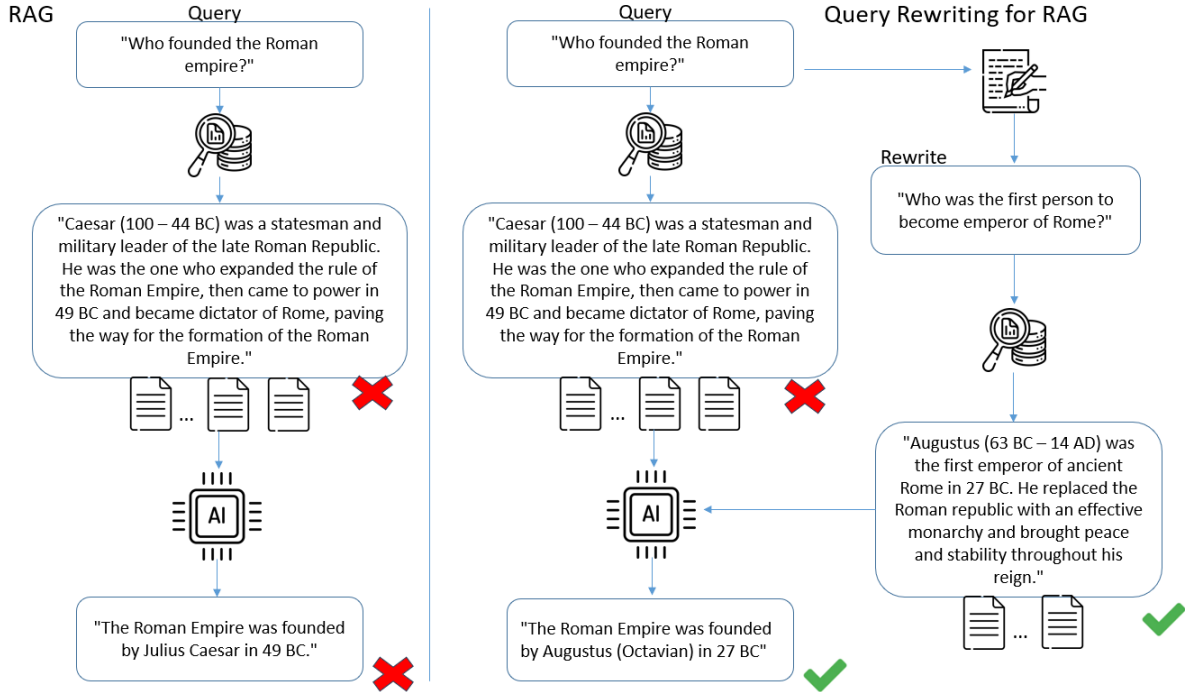


Figure 2.4: Query rewriting process in RAG

Embedding transformation: involves either pretraining language models or leveraging pretrained models to gain a deep understanding of the structural and semantic characteristics of text data. This process enables language models to map both queries and structured texts into one unified embedding space, facilitating efficient and accurate retrieval [28]. By doing so, the model enables efficient and accurate retrieval by ensuring that semantically related information from different formats is aligned within the same representation space. This unified embedding not only improves retrieval precision but also facilitates the integration of heterogeneous knowledge sources into the retrieval process.

2.1.5 Retrieval

Once the embedding process is complete, a document retrieval system identifies and returns the most relevant documents based on the query. In a RAG pipeline, this document retrieval step plays a crucial role, supplying the LLM with the appropriate documents for generating contextually accurate responses. Formally, given a query q in an arbitrary language x , it can retrieve document d in language y (which can be another language or the same language as x) from the corpus D^y . Moreover, the retrieval function $fn^*(\cdot)$ belongs to any of the functions: dense, lexical, or multi-vector retrieval [8].

$$d^y \leftarrow fn^*(q^x, D^y)$$

- **Dense Retrieval**

Query q is transformed into the hidden states H_q based on a text encoder.

The normalized hidden state of the special token "[CLS]" is used for the representation of the query:

$$e_q = \text{norm}(H_q[0])$$

Similarly, the embedding of passage p is:

$$e_p = \text{norm}(H_p[0])$$

The relevance score between query and passage is measured by the inner product between the two embeddings e_q and e_p

$$s_{dense} \leftarrow \langle e_p, e_q \rangle$$

- **Lexical Retrieval**

For each term $\mathbf{t}$ in the query:

$$w_{q_t} \leftarrow \text{ReLU}(W_{lex}^T H_q[i])$$

Similarly, for the passage:

$$w_{p_t} \leftarrow \text{ReLU}(W_{lex}^T H_p[i])$$

Where:

- $W_{lex} \in \mathbb{R}^{d \times 1}$: A weight matrix mapping the hidden state to a scalar.
- $H_q[i], H_p[i]$: Hidden states (embedding vectors) for the i -th token in the query and passage.
- ReLU : Rectified Linear Unit activation function, defined as $\text{ReLU}(x) = \max(0, x)$.

- **Multi-vector Retrieval**

Representing query and passage with multiple vectors

Formulas:

- Query Representation:

$$E_q = \text{norm}(W_{mul}^\top H_q)$$

- Passage Representation:

$$E_p = \text{norm}(W_{mul}^\top H_p)$$

- $W_{mul} \in \mathbb{R}^{d \times d}$: A learnable projection matrix.
- H_q and H_p : Hidden states (embeddings) for the query and passage, respectively.
- norm : Column-wise normalization function applied to each vector in the matrices.

2.1.6 Post-Retrieval

Reranking: The initial retrieval stage often relies on methods like dense or lexical retrieval, which are efficient but may not consistently yield highly relevant documents. To address this, reranking plays a crucial role by reorganizing the retrieved documents to better align with the given query, thereby improving their relevance.

Contextual Compression: Contextual compression addresses the challenge of retrieving relevant information from large documents by compressing and filtering content based on the query. Instead of passing entire documents, which can lead to expensive and less effective responses, this approach refines

the retrieved results. Using a base retriever and a Document Compressor tools like Compact [58], it will shorten documents by extracting only the relevant content or removing irrelevant ones entirely, ensuring more efficient and focused query responses.

2.2 Generator

The posed query and selected documents are synthesized into a coherent prompt to which a large language model is tasked with formulating a response. The model’s approach to answering may vary depending on task-specific criteria, allowing it to either draw upon its inherent parametric knowledge or restrict its responses to the information contained within the provided documents. In cases of ongoing dialogues, any existing conversational history can be integrated into the prompt, enabling the model to engage in multi-turn dialogue interactions effectively [14].

In this section, we provide an overview of the architecture of Transformer models [49] and delve into their core mechanisms that have revolutionized natural language processing. Following this, we explore various prompt engineering techniques, highlighting their role in optimizing the performance of pre-trained language models across a wide range of tasks, including RAG. These techniques not only improve the model’s understanding of task-specific contexts but also enhance its ability to generate accurate and contextually relevant responses.

2.2.1 Transformer

Transformer [49], a model architecture eschewing recurrence and instead relying entirely on an attention mechanism to draw global dependencies between input and output [49]. It has revolutionized natural language processing (NLP) by enabling parallel processing of sequences. Unlike recurrent architectures such as RNNs [41] or LSTMs [17], which process inputs sequentially and struggle with long-range dependencies, Transformers use self-attention mechanisms to focus on relevant parts of the input across the entire sequence simultaneously. This approach not only improves the ability to capture intricate relationships within the data but also drastically increases computational efficiency by enabling systems to handle tasks in parallel. The result is a model architecture that excels at tasks requiring a deep understanding of context, from machine translation to text generation, setting the foundation for state-of-the-art language models like GPT [34], BERT [11], and T5 [36].

Encoder and Decoder Stacks: A Transformer consists of two components: Encoder and Decoder. Each of them is a stack of N identical layers (N=6 in the original Transformer model).

- **Encoder:** The Encoder is composed of a stack of N identical layers. Each layer has two sub-layers. The first is a multi-head self-attention mechanism, and the second is a simple, position-wise fully connected feed-forward network. Around each of the two sub-layers, there is a residual connection [15] followed by layer normalization [2]. That is, the output of each sub-layer is $\text{LayerNorm}(x + \text{Sublayer}(x))$, where $\text{Sublayer}(x)$ is the function implemented by the sub-layer itself [49].
- **Decoder:** The Decoder, like the Encoder, is also composed of a stack of N identical layers. In addition to the two sub-layers in each Encoder layer, the Encoder inserts a third sub-layer, which performs multi-head attention over the output of the Encoder Stack. The self-attention sub-layer in the decoder stack is modified to prevent positions from attending to subsequent positions. This masking, combined with fact that the output embeddings are offset by one position, ensures that the predictions for position i can depend only on the known outputs at positions less than i [49].

Positional Encoding: plays a critical role in capturing the order of sequences, which is essential for preserving the structural integrity of text data. Positional Encodings are added to the input embeddings at the initial layers of both the Encoder and Decoder stacks, enabling the model to incorporate sequential information.

In the original model, sine and cosine functions of different frequencies is used for Positional Encoding:

$$\text{PE}(\text{pos}, 2i) = \sin(\text{pos}/10000^{2i/d_{\text{model}}}) \quad (2.1)$$

$$\text{PE}(\text{pos}, 2i + 1) = \cos(\text{pos}/10000^{2i/d_{\text{model}}}) \quad (2.2)$$

where pos is the position and i is the dimension. That is, each dimension of the positional encoding corresponds to a sinusoid [49].

Self-Attention Mechanism:

- **Queries, Keys and Values:** In self-attention mechanism, Transformer compute output by mapping a query with a set of key-value pairs. The output is computed as a weighted sum of the values, where the weight computed by scaled dot-product of query and keys.
- **Scaled-dot products:** The concept of self-attention lies in computing a weighted sum of values, where the weights represent the similarity between queries and keys. In Transformers, this similarity is determined using a dot product between the query and key vectors, followed by a softmax function. The softmax operation normalizes these weights, ensuring they fall within the range of 0 and 1, thus effectively controlling their influence.
- The vectors for queries, keys, and values are all of size d_K . When computing the dot product between queries and keys, the resulting values can become disproportionately large, especially as d_K increases. Such large values can cause gradients to vanish during backpropagation, hindering the training process. To mitigate this, Transformers introduce a scaling factor $\sqrt{d_K}$, dividing the dot product by this value. This adjustment helps stabilize the gradients and ensures the training remains efficient and effective, even with large vector dimensions.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_K}}V\right) \quad (2.3)$$

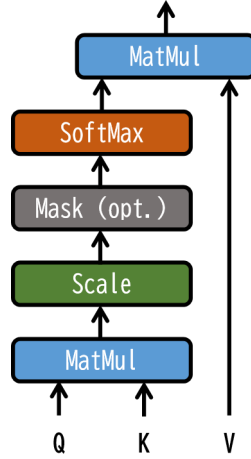


Figure 2.5: Scaled Dot-Product Attention [49].

- **Multi-head attention:** Instead of performing a single attention function with keys, values and queries, it is good to linearly project the queries, keys and values h times with different, learned linear projections to d_k , d_k and d_v dimensions, respectively. On each of these projected versions of queries, keys and values we then perform the attention function in parallel. Here, $W_i^Q, W_i^K \in R^{d_{model} \times d_v}$, $W_i^Q \in R^{d_{model} \times d_v}$, $W^O \in R^{h d_v \times d_{model}}$ and h is the number of attention heads.

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h)W^O \\ \text{where } \text{head}_i &= \text{Attention}(QW_i^Q, KW_i^K, VW_i^V) \end{aligned} \quad (2.4)$$

2.2.2 Prompt engineering

Prompt engineering is an essential methodology for maximizing the potential of pre-trained language models. It focuses on carefully crafting task-specific instructions, known as prompts, to direct the model's output effectively without requiring any modifications to its underlying parameters [43]. Various techniques have been developed to refine the practice of prompt engineering, each aimed at leveraging the full potential of pre-trained language models for diverse and complex applications. Among the most prominent methods are:

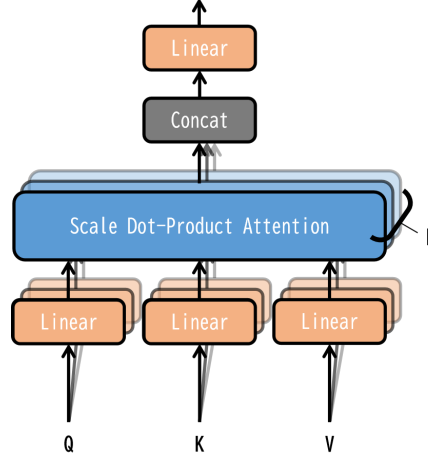


Figure 2.6: Multi-Head Attention (consists of several attention layers running in parallel) [49].

- **Zero-Shot (0S) prompting:** This technique involves providing the model with a clear and specific instruction for a task without any prior examples. This method provides maximum convenience, potential for robustness, and avoidance of spurious correlations [6]. Zero-shot prompting relies entirely on the model’s pre-trained knowledge and ability to generalize across contexts, making it effective for a wide range of tasks without additional data.
- **Few-Shot (FS) prompting:** This method works by giving K examples (with K often ranging from 10 to 100) of context and completion, and then one final example of context, then the model is expected to provide the final completion [6]. These K examples act as a guide for the model, helping it understand the structure, context, or desired behavior for the task.
- **One-Shot (1S) prompting:** This approach is the same as FS Prompting but only one example is used for the prompt. This way of prompting closely matches the way in which some tasks are communicated to humans [6], normally we are given one single example or demonstration of the task then we are asked to complete other similar tasks.
- **Chain-of-Thought (CoT) prompting:** This technique involves prompting a coherent series of intermediate reasoning steps that lead to the final answer for a problem rather than providing a direct answer [55]. By simulating a logical progression of thought, the model is better equipped to handle complex tasks that require multi-step reasoning, such as mathematical problem-solving or logical deductions. This structured guidance fosters greater accuracy and depth in responses.

2.3 Augmentation methods

2.3.1 Augmentation stage: Inference

At the inference stage, augmentation involves retrieving and integrating external knowledge dynamically during the generation process. Recent work, such as In-Context RALM [37], demonstrates the efficacy of prepending retrieved documents to the input text without modifying the language model (LM) architecture. This approach leverages off-the-shelf retrievers for document selection, providing significant performance gains while maintaining simplicity.

Language models define probability distributions over sequences of tokens. Given such a sequence $x_1, \dots, x_n$, the standard way to model its probability is via next-token prediction:

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i | x_{<i}),$$

where $x_{<i} := x_1, \dots, x_{i-1}$ is the sequence of tokens preceding x_i , also referred to as its prefix. This autoregressive model is usually implemented via a learned transformer network [49] parameterized by the set of parameters θ :

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p_\theta(x_i | x_{<i}).$$

Retrieval-augmented language models (RALMs) add an operation that retrieves one or more documents from an external corpus C , and conditions the above LM predictions on these documents. Specifically, for predicting x_i , the retrieval operation from C depends on its prefix $RC(x_{<i})$, giving rise to the general RALM decomposition:

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p_\theta(x_i | x_{<i}, RC(x_{<i})).$$

In-context RALM refers to a simple method of concatenating the retrieved documents within the transformer’s input before the prefix, without altering the LM weights θ :

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p_\theta(x_i | [RC(x_{<i}); x_{<i}]),$$

where $[a; b]$ denotes the concatenation of strings a and b . This approach allows the model to incorporate external knowledge efficiently.

Two practical design choices in RALM systems significantly impact performance:

1. **Retrieval Stride** Retrieval operations can be performed once every $s > 1$ tokens to reduce computation costs. The retrieval stride s determines how often new documents are fetched, trading runtime for performance. The formulation becomes:

$$p(x_1, \dots, x_n) = \prod_{j=0}^{\lfloor n/s \rfloor - 1} \prod_{i=1}^s p_\theta(x_{s \cdot j + i} | RC(x_{\leq s \cdot j}); x_{<(s \cdot j + i)}).$$

2. **Retrieval Query Length** The retrieval query at stride j is typically restricted to the last ℓ tokens of the prefix ($p_j^{s, \ell} := x_{s \cdot j - \ell + 1}, \dots, x_{s \cdot j}$) to avoid diluting the relevance of the information. The resulting formulation is:

$$p(x_1, \dots, x_n) = \prod_{j=0}^{\lfloor n/s \rfloor - 1} \prod_{i=1}^s p_\theta(x_{s \cdot j + i} | [RC(q_j^{s, \ell}); x_{<(s \cdot j + i)}]).$$

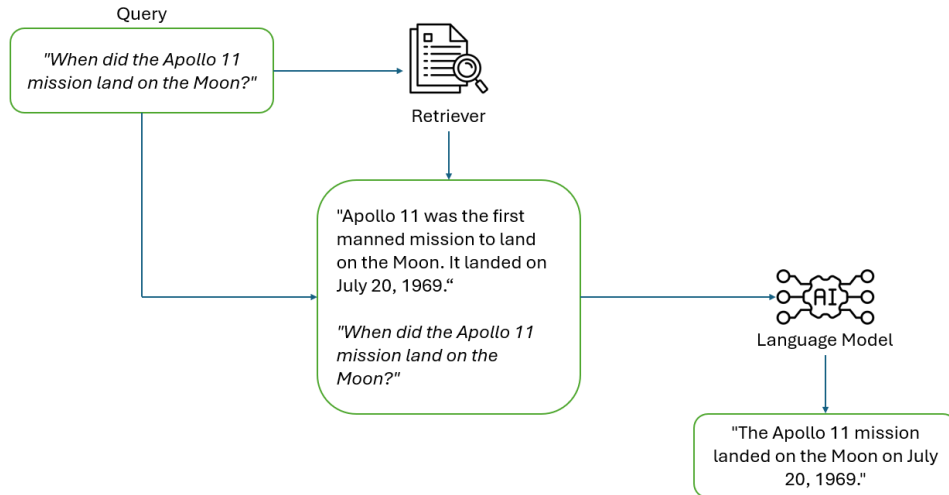


Figure 2.7: Augmentation during the inference stage. Relevant documents are retrieved and prepended to the input for improved factual grounding.

2.3.2 Augmentation source: Structured data

Structured data, such as knowledge bases (KBs), offers a rich source for augmentation. The KnowledGPT framework [53] demonstrates how LLMs can interact with KBs for both retrieval and storage. By generating programmatic queries, LLMs can navigate multi-hop and entity-specific questions effectively.

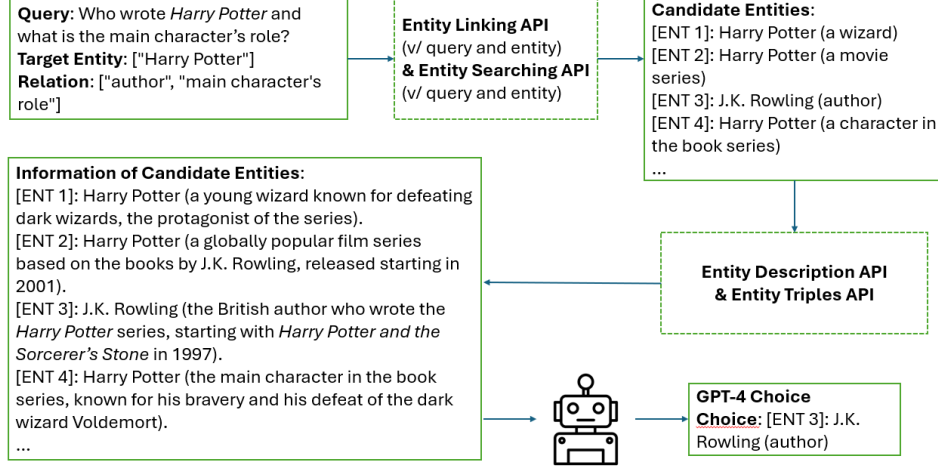


Figure 2.8: Workflow of augmentation with structured data sources [53].

To enable effective interaction with structured data, KnowledGPT employs several key functions:

- *get_entity_info*: Retrieves encyclopedic descriptions of an entity from the KB.
- *find_entity_or_value*: Extracts entities or attribute values related to a query by identifying the most relevant relation.
- *find_relationship*: Identifies relationships between two entities within the KB.

These functions allow KnowledGPT to leverage the structured nature of KBs to extract precise and relevant knowledge.

The *find_entity_or_value* function is central to querying entities and attribute values. It matches the query's relation to KB triples using embedding-based similarity and retrieves the corresponding entities or values. The algorithm is detailed below.

2.3.3 Augmentation process

The augmentation process in RAG frameworks can be categorized into various types, such as Once, Iterative, Recursive, and Adaptive. Among these, the Self-Reflective Retrieval-Augmented Generation (SELF-RAG) [1] approach employs an adaptive process where the model dynamically determines the necessity of retrieval, evaluates the relevance and support of retrieved passages, and critiques its own output for factuality and quality.

Reflection tokens are special tokens used by SELF-RAG to evaluate and guide the retrieval and generation process.

Type	Input	Output	Definitions
Retriever	$x/x, y$	{yes, no, continue}	Decide to Retrieve with $\mathcal{R}$
IsREL	x, d	{ relevant , irrelevant }	d provide useful information to solve x
IsSUP	x, d, y	{ fully supported , partially supported, no support}	All of the verification-worthy statement in y is supported by d
IsUSE	x, y	{5, 4, 3, 2, 1}	y is useful response to x

Table 2.1: Reflection tokens [1].

The inference process in SELF-RAG leverages reflection tokens to dynamically retrieve relevant passages and critique the model's outputs. The steps are as follows:

Algorithm 6 find_entity_or_value

Input: query q , alias list of entity $\mathcal{E}$, alias list of relation $\mathcal{R}$.

Output: a list of target entities or attribute values $\mathcal{T}$.

```
1: function EMBSIM(str  $r$ , list[str]  $\mathcal{R}$ )
2:    $s = -1$ 
3:    $r = \text{embedding}(r)$ 
4:   for  $r_i \in \mathcal{R}$  do
5:      $r_i = \text{embedding}(r_i)$ 
6:     if  $\cos(r, r_i) > s$  then
7:        $s = \cos(r, r_i)$ 
8:     end if
9:   end for
10:  return  $s$ 
11: end function
12:  $e = \text{entity\_linking}(\mathcal{E})$ 
13: if  $e == \text{NULL}$  then
14:   return  $\text{NULL}$ 
15: end if
16:  $r = \text{NULL}$ 
17:  $s_r = -1$ 
18: for  $\text{triple} \in \text{triples}$  do
19:    $r_i = \text{triple.rel}$ 
20:    $s_i = \text{EMBSIM}(r_i, \mathcal{R})$ 
21:   if  $s_i > s_r$  then
22:      $s_r = s_i$ 
23:      $r = r_i$ 
24:   end if
25: end for
26: if  $r == \text{NULL}$  then
27:   return  $\text{NULL}$ 
28: end if
29:  $\text{triples}_r = \text{triples}$  with relation  $r$ 
30:  $\mathcal{R} =$  target entities or attribute values in  $\text{triples}_r$ 
31: return  $\mathcal{R}$ 
```

1. **Input and Prediction:** The model receives the input prompt x and preceding generation $y < t$. It predicts whether retrieval is necessary using the Retrieve token.
2. **Retrieve:** If retrieval is needed (Retrieve = yes), the retriever fetches relevant passages D based on the input and context.
3. **Generate and Evaluate:** For each retrieved passage $d \in D$, the model:
 - Predicts IsREL to assess relevance.
 - Generates the next segment y_t .
 - Predicts IsSUP and IsUSE to evaluate support and utility.
4. **Rank and Select:** The model ranks all generated segments based on IsREL, IsSUP, and IsUSE, and selects the top-ranked segment.
5. **No Retrieval Case:** If retrieval is not needed (Retrieve = no), the model directly generates the next segment and predicts its utility.
6. **Iterative Refinement:** The process repeats until the response is complete.

Algorithm 7 SELF-RAG Inference

Require Generator LM $\mathcal{M}$, Retriever $\mathcal{R}$, Large-scale passage collections $\{d_1, \dots, d_N\}$

- 1: **Input:** input prompt x and preceding generation $y_{<t}$, **Output:** next output segment y_t
 - 2: $\mathcal{M}$ predicts Retriever given $(x, y_{<t})$
 - 3: **if** Retriever == Yes **then**
 - 4: Retrieve relevant text passages $\mathcal{D}$ using $\mathcal{R}$ given (x, y_{t-1}) ▷ Retrieve
 - 5: $\mathcal{M}$ predicts IsREL given x, d and $y_{<t}$ for each $d \in \mathcal{D}$ ▷ Generate
 - 6: $\mathcal{M}$ predicts IsSUP and IsUSE given x, d for each $d \in \mathcal{D}$ ▷ Critique
 - 7: Rank y_t based on IsREL, IsSUP, IsUSE
 - 8: **else if** Retriever == No **then**
 - 9: $\mathcal{M}_{\text{gen}}$ predicts y_t given x ▷ Generate
 - 10: $\mathcal{M}_{\text{gen}}$ predicts IsUSE given x, y_t ▷ Critique
 - 11: **end if**
-

3.1 RAG and LLMs in Coding Tasks

3.1.1 Retrieval Strategies

Sparse Retrieval Methods: Tools like BM25 [5] rank documents based on term frequency and inverse document frequency (TF-IDF). While highly effective for small-scale knowledge bases, their efficiency declines as data scales increase [16].

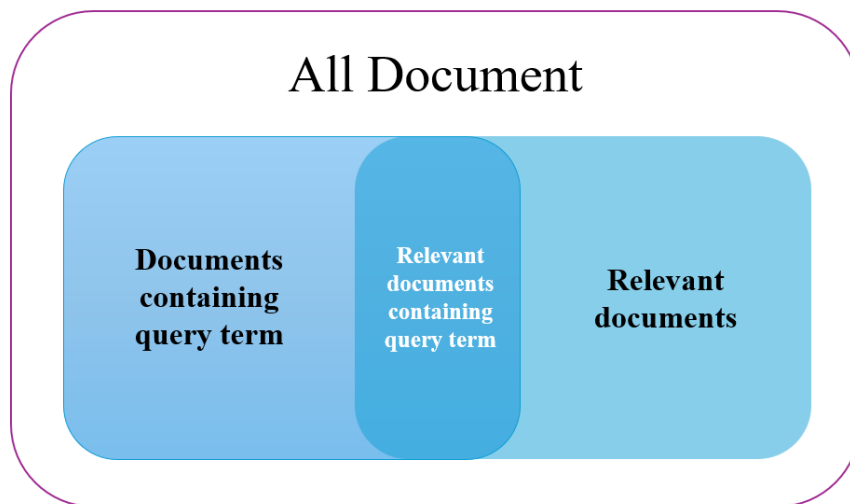


Figure 3.1: BM25 visualization.

Dense Retrieval Methods: Leveraging semantic embeddings, dense retrieval tools like SBERT [38]’s Semantic Search and approximate techniques (e.g., HNSW [29], ANNOY [4], and LSH [21]) provide scalable solutions for large knowledge bases. These approaches achieve significant speed improvements with minimal accuracy trade-offs [16].

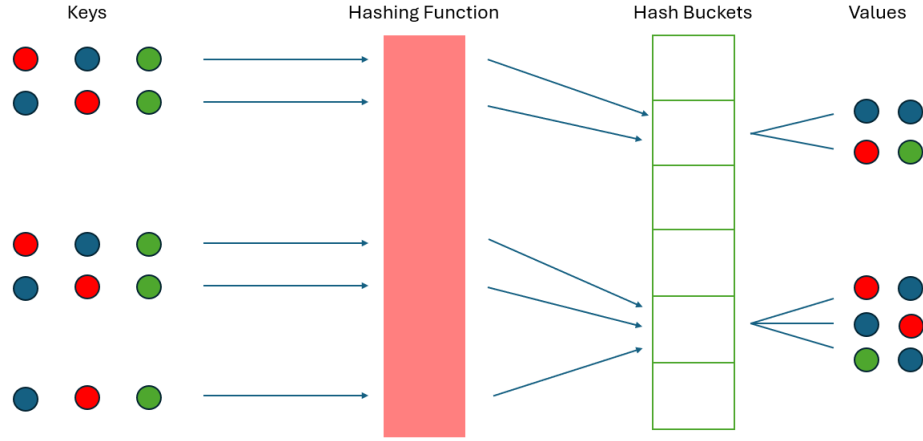


Figure 3.2: LSH visualization.

Selective Retrieval: Frameworks such as Repoformer [56] employ selective retrieval, determining the necessity of retrieval based on contextual evaluation. This approach reduces inefficiencies and avoid adding irrelevant context.

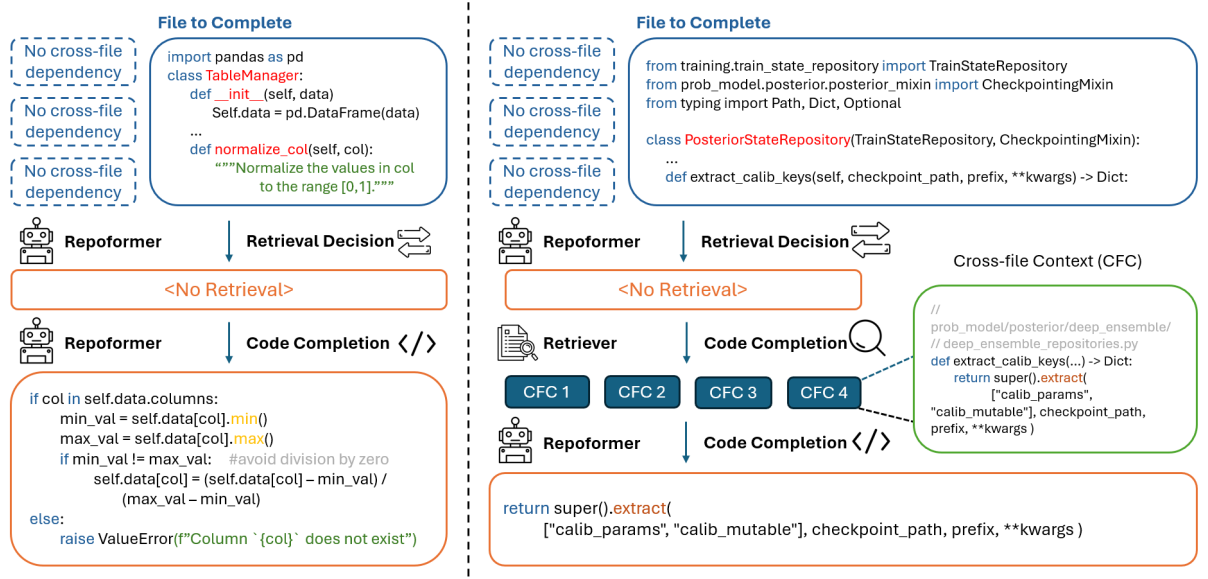


Figure 3.3: REPOFORMER framework [56].

Multi-Retrieval Perspectives: Systems like ProCC [45] utilize diverse perspectives, including lexical semantics, hypothetical line embeddings, and code summarization, to enrich context and improve code completion accuracy.

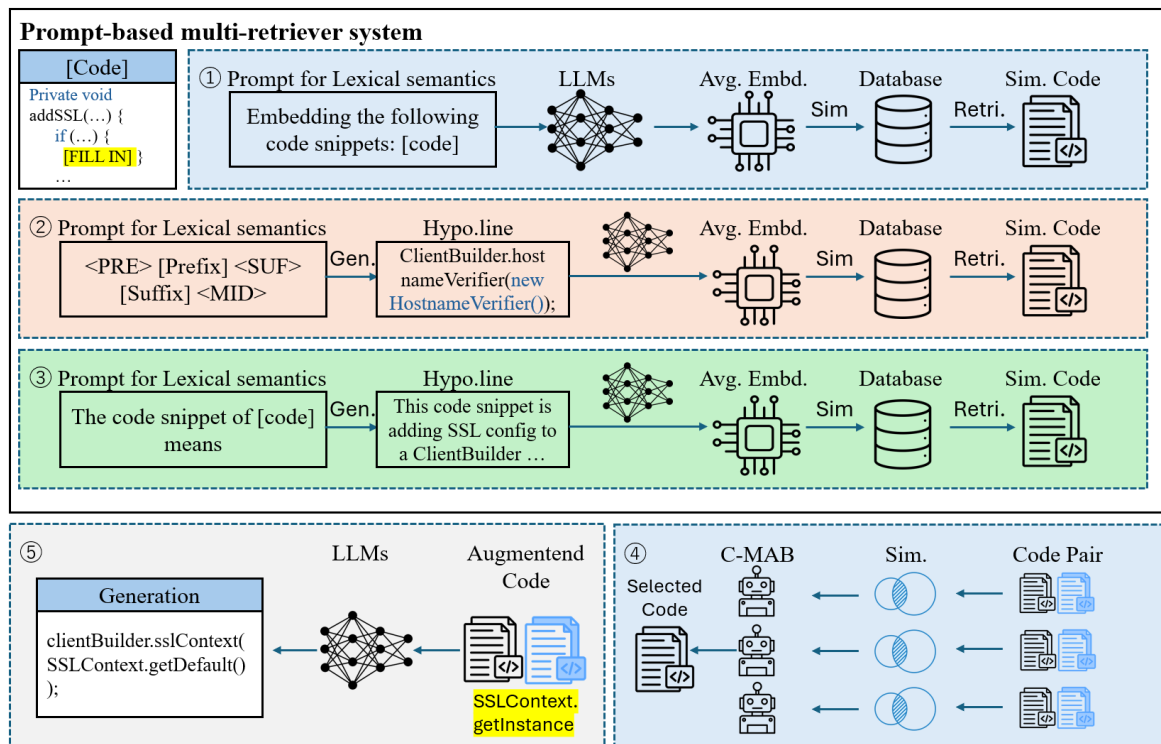


Figure 3.4: ProCC framework [45].

Multilingual and Multifunctional Embeddings: Models like BGE M3-Embedding [8] support over 100 languages and various retrieval functionalities, including dense, multi-vector, and sparse retrieval, enhancing versatility in coding tasks.

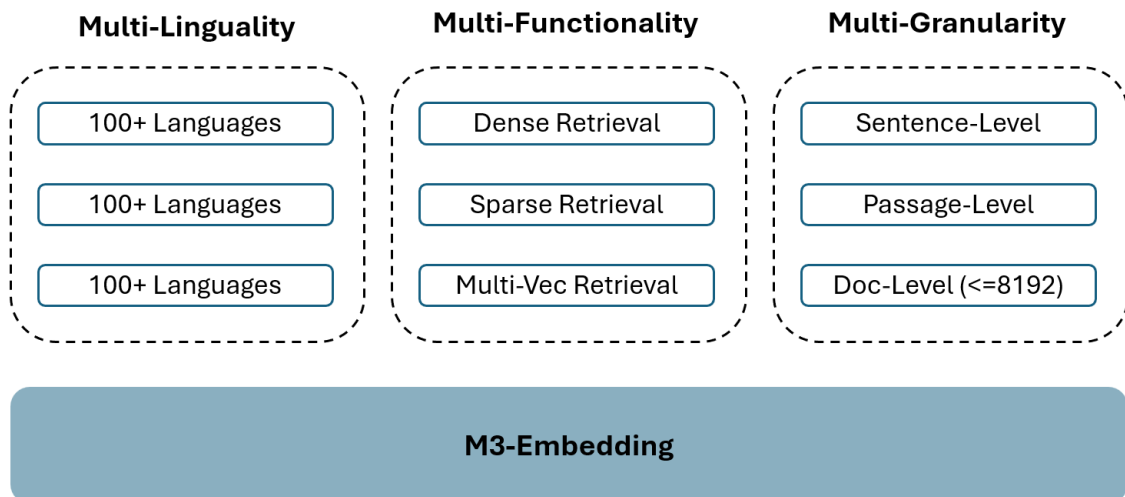


Figure 3.5: Characters of M3-Embedding [8].

3.1.2 Generative Enhancements

Generative Pre-trained Transformers: LLMs like GPT-2 [35] and Code Llama [40] are trained on extensive code corpora, enabling them to handle diverse tasks such as function completion and API usage.

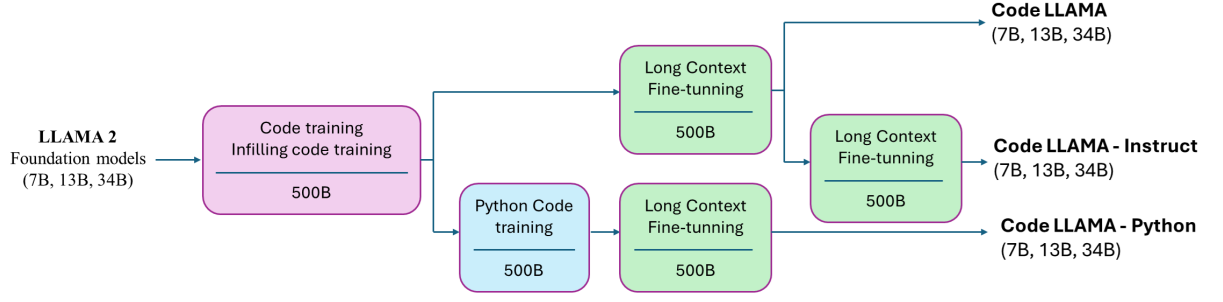


Figure 3.6: The Code Llama specialization pipeline [40].

In-Context Augmentation: RAG pipelines integrate retrieved code snippets directly into prompts, enhancing generation without additional fine-tuning. This approach is particularly effective for repository-level code generation [18].

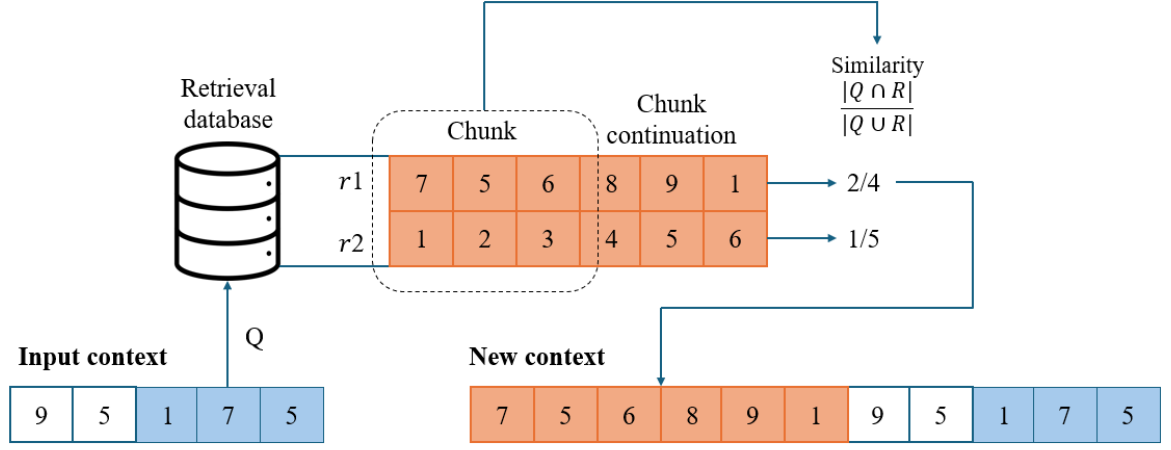


Figure 3.7: The retrieved snippet contains continuation tokens and additional metadata [18].

Specialized Models for Coding: Models like Qwen2.5-Coder [19] and CodeQwen1.5-7B-Chat [3] are fine-tuned for coding tasks, demonstrating improved performance in code generation and comprehension.

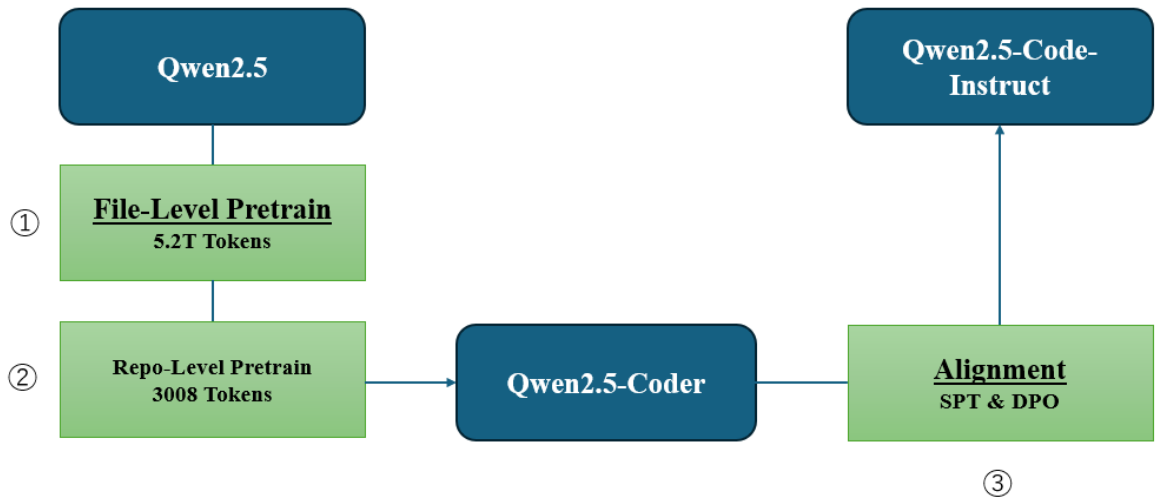


Figure 3.8: The three-stage training pipeline for Qwen2.5-Coder [18].

3.1.3 Specialized Applications

Repository-Level Code Completion: Tools like Repoformer and RepoCoder [59] focus on cross-file dependencies, retrieving and integrating relevant code snippets to improve repository-wide coherence.

<pre># Below are some referential code fragments # from other files: # ----- # the below code fragment can be found in # tests/test_pipelines_common.py # ----- # @unittest.skipIf(torch_device != "cuda") # def test_to_device(self): # components = self.get_dummy_components() # pipe = self.pipeline_class(**components) # pipe.progress_bar(disable=None) # pipe.to("cpu") # ----- """Based on above, complete the following code:"""</pre>	# Retrieved Code
<pre>@unittest.skipIf(torch_device != "cuda") def test_float16_inference(self): components = self.get_dummy_components()</pre>	# Unfinished Code
<pre>pipe = self.pipeline_class(**components) pipe.to(torch_device)</pre>	# Model Prediction

Figure 3.9: Demonstrating the format of the RepoCoder prompt [59].

Commit Message and Assertion Generation: RAG systems generate commit messages and assertions by retrieving semantically aligned examples, balancing efficiency and effectiveness [16].

Code Hallucination Mitigation: Automated code generation, powered by LLMs, has significantly enhanced development efficiency by generating code from input requirements. However, LLMs often produce hallucinations, particularly when handling complex contextual dependencies in repository-level development scenarios. To address these challenges, RAG-based mitigation methods have been proposed [62].

3.2 Developer Assistance Tools

3.2.1 Functionality

Search and Retrieval Tools: Platforms like Stack Overflow¹ provide a database of Q&A posts where users search for solutions based on keywords or queries. Neural Code Search (NCS) [42] enhances this by enabling natural language queries over raw codebases without curated answers, using vector embeddings and ranking strategies.

¹Stack Overflow - <https://stackoverflow.com/>

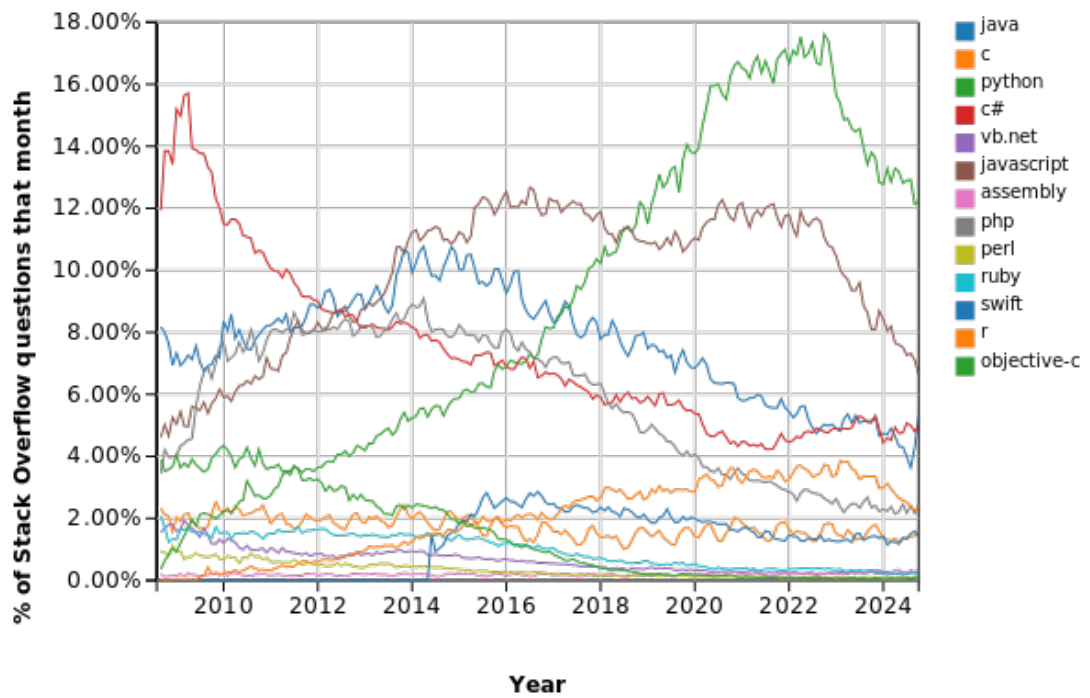


Figure 3.10: Stackoverflow trends chart 2008-2024.

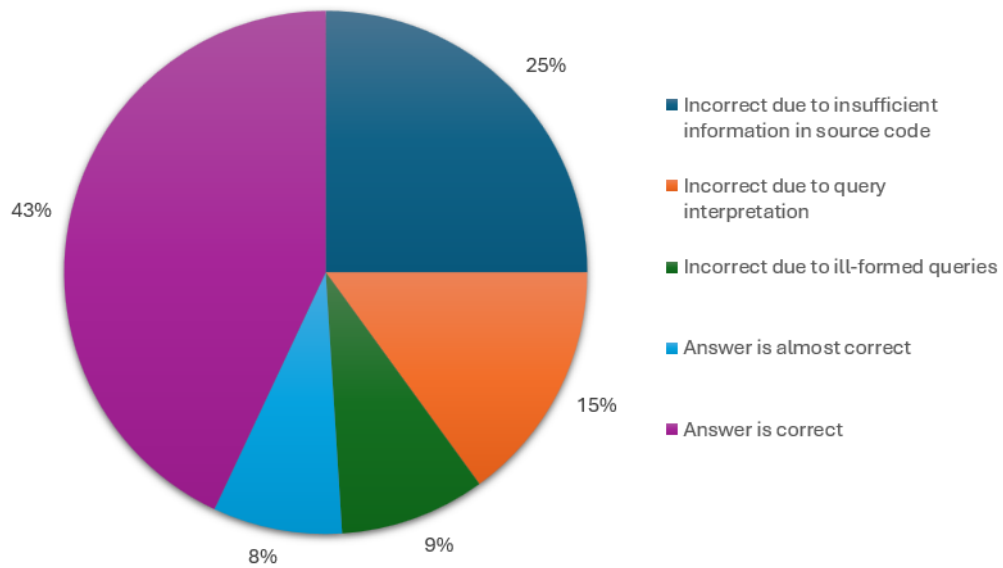


Figure 3.11: NCS evaluation results [42].

Code Completion and Suggestion Systems: Intelligent tools like IntelliCode Compose [44] and example-based systems [7] prioritize providing relevant suggestions for method calls, arguments, and entire lines of code. These systems are designed to enhance developer productivity by learning from existing codebases.

Hybrid Generation Systems: RAG models combine retrieval with generative capabilities [25], producing specific and contextually appropriate outputs by accessing external document repositories and leveraging models

3.2.2 Retrieval Mechanisms

Keyword-Based Retrieval: Traditional forums rely on keyword searches, often yielding broad results with low contextual relevance.

Embedding-Based Retrieval: Tools like NCS employ word embeddings, TF-IDF weighting [31], and vector similarity for precise retrieval of relevant code snippets. These systems also incorporate syntactic and semantic elements extracted from source code.

Rule-Based Retrieval: Example-based systems mine patterns and associations (e.g., frequent co-occurrence of API calls) from repositories to recommend contextually relevant options.

3.2.3 Code Generation Capabilities

Static Suggestion Systems: Earlier systems [7] relied solely on static type information, offering alphabetically sorted method recommendations without understanding contextual relevance.

Generative Models: IntelliCode Compose represents a generative approach, using transformer-based architectures [49] like GPT-C to predict and generate entire lines or blocks of syntactically correct code. This includes multilingual support and optimized decoding methods like beam search.

3.2.4 Integration in Development Processes

Standalone Search Interfaces:

- **NCS** [42]: Functions as an external platform where developers input natural language queries to retrieve relevant code snippets directly from large codebases.
- **Sourcegraph**²: Provides a universal code search and navigation tool, enabling developers to search across multiple repositories and languages from a centralized interface.
- **Krugle**³: A specialized search engine designed for searching open-source and enterprise code repositories. It indexes and organizes code by context, allowing developers to locate relevant examples, API usage, and documentation efficiently.

Feature	NCS	Sourcegraph	Krugle
Functionality	Retrieves code snippets from large codebases using natural language queries.	Provides universal code search and navigation across multiple repositories and languages.	Searches open-source and enterprise code repositories, indexing code by context.
Search Methodology	Maps natural language queries and code snippets into a shared vector space using neural networks.	Offers structural code search, matching nested expressions and code blocks beyond regular expressions.	Indexes code with advanced filtering for examples, API usage, and documentation.
Supported Languages	Initially focused on Java.	Supports over 30 programming languages.	Supports multiple programming languages.
Deployment Options	Research project with datasets available for evaluation.	Offers self-hosted solutions via Docker Compose and Kubernetes, as well as a managed cloud service.	Web-based platform.

Table 3.1: Comparison of Neural Code Search (NCS), Sourcegraph, and Krugle

IDE-Embedded Assistance:

- **IntelliCode Compose** [44]: Integrates into VSCode, offering real-time code generation and suggestions.
- **Github Copilot**⁴: Embedded within various IDEs, including VSCode and JetBrains, it leverages AI to provide code completions and suggestions based on the current context.

²Sourcegraph - <https://sourcegraph.com/>

³Krugle - <https://www.krugle.com/>

⁴Github Copilot - <https://github.com/features/copilot>

- **Tabnine**⁵: An AI-powered code completion tool that integrates with various IDEs, offering whole-line and full-function code completions based on the current code context.

Feature	IntelliCode Compose	GitHub Copilot	Tabnine
Functionality	Provides real-time code generation and suggestions within VSCode.	Offers AI-driven code completions and suggestions across various IDEs.	Delivers whole-line and full-function code completions based on current code context.
Supported IDEs	VSCode.	VSCode, JetBrains suite, Neovim, and others.	VSCode, IntelliJ IDEA, Sublime Text, Atom, and more.
AI Model	Utilizes machine learning models trained on open-source GitHub repositories.	Powered by OpenAI's Codex model, trained on a vast dataset of public code.	Employs proprietary models trained on permissively licensed open-source code.
Privacy and Security	Processes code locally, enhancing privacy.	Processes code in the cloud, which may raise privacy concerns for sensitive codebases.	Offers both cloud-based and on-premises solutions, with options for local processing to ensure code privacy.
Pricing	Included with VSCode.	Individual model: \$10/month; Business version at \$19/month and Enterprise at \$39/user/-month	Basic model: free version with basic features; Dev version at \$9/month and Enterprise at \$39/user/-month

Table 3.2: Comparison of IntelliCode Compose, GitHub Copilot, and Tabnine

```

1 # From: https://pytorch.org/tutorials/beginner/blitz/cifar10_tutorial.html
2 import torch
3 import torchvision
4 import torchvision.transforms as transforms
5
6 transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5, 0.5,
7                                     0.5), (0.5, 0.5, 0.5))])
8
9 trainset = torchvision.datasets.CIFAR10(root='./data', train=True,
10                                         download=True, transform=transform)
11 trainloader = torch.utils.data.DataLoader(trainset, batch_size=4,
12                                           shuffle=True, num_workers=2)
13
14 testset = torchvision.datasets.CIFAR10(root='./data', train=False,
15                                         download=True, transform=transform)
16 testloader = torch.utils.data.DataLoader(testset, batch_size=1, shuffle=False,
17                                           num_workers=2)

```

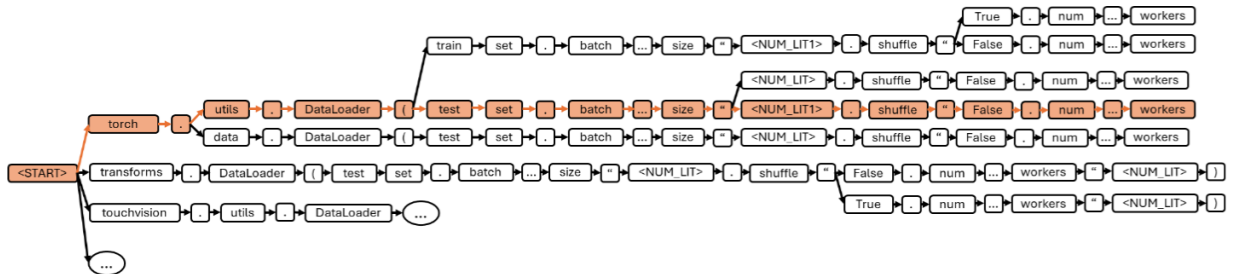


Figure 3.12: Code Snippet and Completion suggestion

⁵Tabnine - <https://www.tabnine.com/>

4.1 Fine-tuning vs. RAG

4.1.1 Fine-tuning

Fine-tuning involves adapting a pre-trained LLM to specific tasks or domains by further training it on a targeted dataset. This process refines the model's parameters to align with specialized requirements, enhancing its performance in particular applications.

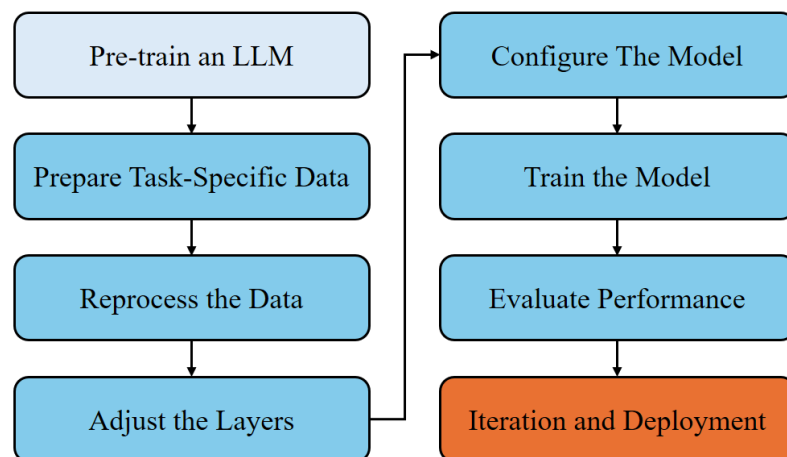


Figure 4.1: Fine-Tuning Process (RAG vs Fine-Tuning: A Comparative Analysis of LLM Learning Techniques¹).

¹<https://addepto.com/blog/rag-vs-fine-tuning-a-comparative-analysis-of-llm-learning-techniques/>

4.1.2 Comparison between fine-tuning and RAG

Aspect	Fine-tuning	RAG
Dynamic vs. Static	Static: Relies on pre-trained knowledge, requiring retraining to update information.	Dynamic: Accesses up-to-date external data sources, providing current information without retraining.
Architecture	Utilizes a pre-trained LLM adjusted with task-specific datasets.	Combines LLM with external knowledge bases, enabling retrieval and generation.
Customization	High customization in domain-specific knowledge, tone, and style.	Limited customization to domain behavior but excels in integrating external content.
Hallucinations	Reduces hallucinations through domain-specific training but may hallucinate with unknown queries.	Minimizes hallucinations by grounding responses in retrieved documents.
Accuracy	High accuracy in specific, trained domains but limited outside the training scope.	Highly accurate for retrieving up-to-date, domain-relevant information.
Transparency	Functions as a "black box" with limited insight into response reasoning.	Transparent response generation with traceable data sources.
Cost	Higher cost due to significant computational resources, labeled data requirements, and periodic retraining for updates.	Lower cost as it eliminates the need for retraining and requires minimal or no labeled data by leveraging external knowledge bases.
Complexity	More complex due to the need for expertise in NLP, deep learning, and hyperparameter tuning.	Simpler to set up with minimal coding for retrieval integration.
Adaptability	Less adaptable, requiring retraining for new information.	Easily adapts to new data and dynamic environments.

Table 4.1: Comparison of Fine-Tuning and Retrieval-Augmented Generation

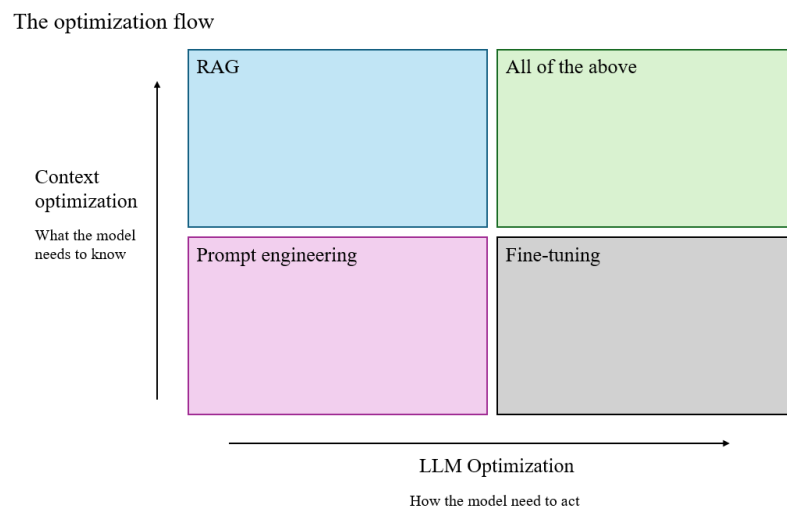


Figure 4.2: The nature of Fine-Tuning and RAG (Fine-tuning vs. RAG: Understanding the Difference²).

4.1.3 Rationale for Choosing RAG

Minimize hallucinations: RAG provides real-time access to the latest documentation and programming guidelines, ensuring that code suggestions are current and precise without the need for extensive retraining.

²<https://finetunedb.com/blog/fine-tuning-vs-rag/>

Transparency: By grounding responses in specific, retrievable documents, RAG offers clear traceability of information sources. This transparency fosters trust in the generated outputs, as developers can verify the origins of the suggestions provided.

4.2 Why BGE-M3?

In RAG systems, the quality of embeddings plays a critical role in determining the relevance and precision of retrieved content. BGE-M3 [8], a state-of-the-art embedding model, has demonstrated superior performance in multilingual retrieval tasks, as shown in Table 4.2. Compared to baseline models such as BM25 [39], mDPR [61], mContriever [20], mE5_{large} [52], E5_{mistral-7b} [51], BGE-M3 achieves an average retrieval score of 71.5 on the MIRACL dataset [60], significantly outperforming prior methods. This score reflects its advanced ability to generate robust and semantically meaningful representations.

The strength of BGE-M3 lies in its ability to unify dense retrieval, sparse retrieval, and multi-vector retrieval techniques. This integration allows it to handle diverse query types with high accuracy and flexibility. Notably, its performance exceeds that of other models in the M3-Embedding framework, with its "All" configuration achieving the highest score by combining the strengths of different retrieval approaches.

In our RAG pipeline, BGE-M3 serves as the foundation for the retrieval stage, ensuring that only the most relevant and semantically precise information is retrieved from the vector database. By leveraging BGE-M3’s advanced capabilities, we aim to enhance the overall effectiveness of the RAG system by improving the quality and relevance of the retrieved context, thus contributing to more accurate and contextually appropriate responses.

Model	Avg Score
Baselines	
BM25	31.9
mDPR	41.8
mContriever	43.1
mE5 _{large}	66.6
E5 _{mistral-7b}	63.4
OpenAI-3	54.9
M3-Embedding	
Dense	69.2
Sparse	53.9
Multi-vector	70.5
Dense+Sparse	70.4
All	71.5

Table 4.2: Comparison of BGE-M3 embeddings with baseline models for multi-lingual retrieval performance on the MIRACL dataset [8].

4.3 Why Qwen?

Qwen [3] is a relatively less recognized yet promising LLM developed by Alibaba Cloud, designed to offer robust natural language understanding and generation capabilities. While its public profile may not be as prominent as some of its contemporaries, the performance metrics in Table 4.3 demonstrate that Qwen is highly competitive in several benchmarks. For instance, the 14B parameter Qwen model achieves state-of-the-art results in tasks such as MMLU (66.3), C-Eval (72.1), and GSM8K (61.3), surpassing many well-known local models like LLaMA [48], Llama2 [47] and Baichuan2 [57]. This highlights Qwen’s strong capability to handle diverse tasks effectively despite having fewer parameters compared to some larger models.

The decision to integrate Qwen into our RAG pipeline stems from its balance of efficiency, performance, and adaptability. By leveraging RAG, we aim to showcase how even a "lighter" LLM like Qwen can achieve substantial improvements by incorporating external knowledge sources. Unlike more resource-intensive models, Qwen’s relatively lightweight architecture allows for efficient local deployment, making it a practical choice for scenarios where computational resources are limited. This approach not only

aligns with our objective of building a lightweight and context-aware system but also emphasizes how RAG techniques can enhance the generative responses and contextual understanding of models that might otherwise be constrained by their architecture.

This comparative analysis provides valuable insights into Qwen’s strengths and limitations, illustrating its potential as an effective and efficient solution when paired with external retrieval mechanisms in RAG-based applications.

Model	Params	MMLU 5-shot	C-Eval 5-shot	GSM8K 8-shot	MATH 4-shot	HumanEval 0-shot	MBPP 3-shot	BBH 3-shot
Baichuan2	7B	54.7	56.3	24.6	5.6	18.3	24.2	41.6
	13B	59.5	59.0	52.8	10.1	17.1	30.2	49.0
LLaMA	7B	35.6	27.3	11.0	2.9	12.8	17.7	33.5
	13B	47.7	31.8	20.3	4.2	15.8	22.0	37.9
	33B	58.7	37.5	42.3	7.1	21.7	30.2	50.0
	65B	63.7	40.4	54.4	10.6	23.7	37.7	58.4
Llama 2	7B	46.8	32.5	16.7	3.3	12.8	20.8	38.2
	13B	55.0	41.4	29.6	5.0	18.9	30.3	45.6
	34B	62.6	42.6	42.2	6.2	22.6	33.0	44.1
	70B	69.8	50.1	63.6	13.5	29.9	45.0	64.9
QWEN	1.8B	44.6	54.7	21.2	5.6	17.1	14.8	28.2
	7B	58.2	63.5	51.7	11.6	29.9	31.6	45.0
	14B	66.3	72.1	61.3	24.8	32.3	40.8	53.4

Table 4.3: Performance comparison of some open-source base models on different benchmarks [3].

4.4 Main architecture

4.4.1 Local Vector Storage

The vector storage component is a fundamental part of our system, enabling efficient retrieval of embedded data for the user. Since our primary dataset consists of documentation from the Next.js framework, it inherently contains a combination of both structured and unstructured data:

- **Structured data:** The Next.js documentation is systematically organized. Each documentation page belongs to a specific subject or category, resembling a hierarchical folder structure. This organization provides metadata that aids in filtering and contextual retrieval.
- **Unstructured data:** The text content and code snippets within the documentation represent unstructured data. These elements lack a predefined schema but carry crucial information necessary for coding assistance.

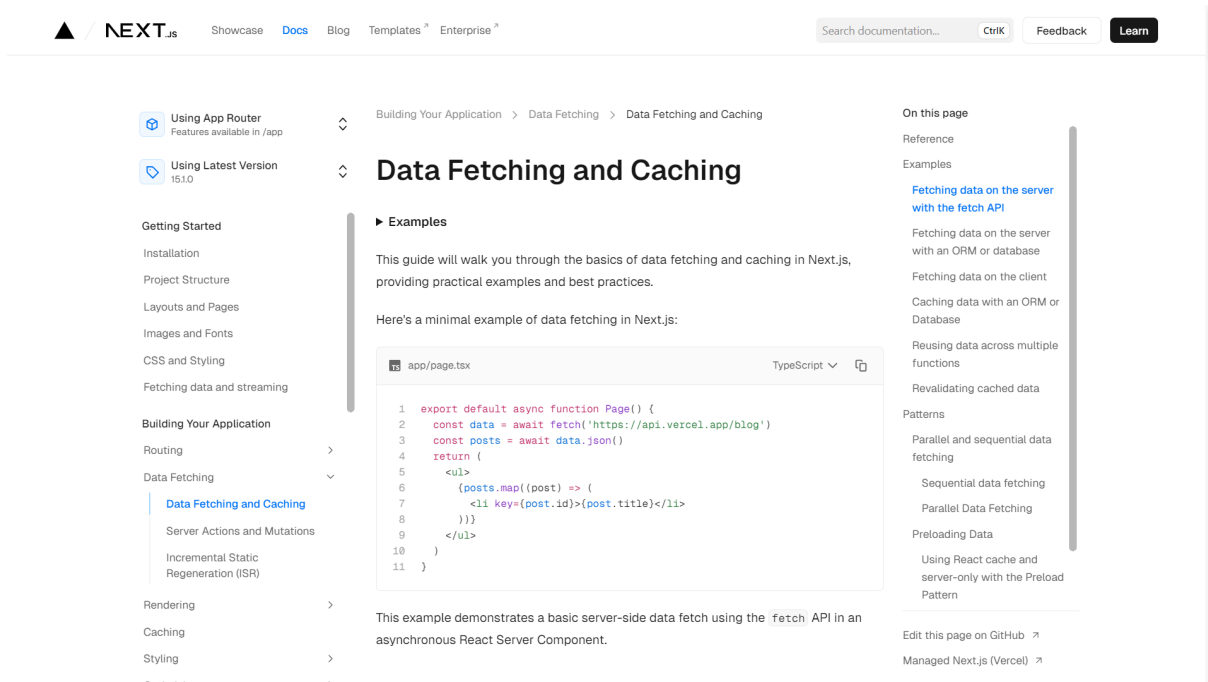


Figure 4.3: Example of a Next.js documentation page.

```

---
title: Data Fetching and Caching
nav_title: Data Fetching and Caching
description: Learn best practices for fetching data on the server or client in
  ↳ Next.js.
---

<details>
  <summary>Examples</summary>

  - [Next.js Commerce] (https://vercel.com/templates/next.js/nextjs-commerce)
  - [On-Demand ISR] (https://on-demand-isr.vercel.app)
  - [Next.js Forms] (https://github.com/vercel/next.js/tree/canary/examples/next-forms)

</details>

This guide will walk you through the basics of data fetching and caching in Next.js,
  ↳ providing practical examples and best practices.

Here's a minimal example of data fetching in Next.js:

```tsx filename="app/page.tsx" switcher
export default async function Page() {
 const data = await fetch('https://api.vercel.app/blog')
 const posts = await data.json()
 return (

 {posts.map((post) => (
 <li key={post.id}>{post.title}
))}

)
}

```

```

...

```jsx filename="app/page.js" switcher
export default async function Page() {
  const data = await fetch('https://api.vercel.app/blog')
  const posts = await data.json()
  return (
    <ul>
      {posts.map((post) => (
        <li key={post.id}>{post.title}</li>
      ))}
    </ul>
  )
}
...

```

This example demonstrates a basic server-side data fetch using the `fetch` API in an asynchronous React Server Component.

Why Milvus?

Milvus is our chosen vector database for storing embedded data in the system. This decision is based on several compelling reasons:

- **Open Source:** Milvus is an open-source database, perfectly aligning with our vision of delivering a free, open-source, distributed code assistance tool for VSCode. Its open nature encourages transparency and community contributions.
- **Support for Structured and Unstructured Data:** The nature of our dataset, comprising both structured metadata (e.g., documentation sections, categories) and unstructured content (e.g., textual descriptions, code snippets), makes Milvus an ideal choice. Structured data benefits from attribute-based filtering, allowing users to narrow their queries to specific documentation sections. Meanwhile, unstructured data, such as free-form text and code snippets, is embedded into high-dimensional vectors for similarity-based retrieval.
- **Comprehensive SDK and Model Support:** Milvus’s SDK seamlessly integrates with our chosen embedding model, BGE-M3, for generating high-quality embeddings. It also supports the HNSW algorithm for efficient indexing of unstructured data. This compatibility with our preferred methods ensures smooth implementation and reliable performance.
- **Scalability:** Milvus employs LSM-based storage to minimize disk I/O and ensure efficient handling of frequent updates. This scalability is critical for managing dynamic and evolving datasets, such as continuously updated framework documentation.
- **Performance:** Since our system is designed to run entirely on the user’s local machine, Milvus’s ability to leverage both CPU and GPU resources significantly enhances performance. This ensures that computationally intensive tasks, such as vector similarity searches and indexing, are executed efficiently, maintaining a responsive user experience.

System	Billion-Scale Data	Dynamic Data	GPU	Attribute Filtering	Multi-Vector Query	Distributed System
Facebook Faiss [12, 22]	✓	✗	✓	✗	✗	✗
Microsoft SPTAG [10]	✓	✗	✓	✗	✗	✓
ElasticSearch [13]	✗	✓	✗	✓	✗	✓
Jingdong Vearch [26, 27]	✓	✓	✓	✓	✗	✓
Alibaba AnalyticDB-V [54]	✓	✓	✗	✗	✗	✓
Alibaba PASE (PostgreSQL) [33]	✗	✓	✗	✓	✗	✓
Milvus [50]	✓	✓	✓	✓	✓	✓

Table 4.4: System comparison.

4.4.2 VSCode integration

The interactive user interface is a crucial part of the system, designed to optimize the developer’s experience while working. This component leverages the power of VSCode Extension to provide seamless integration between the development tool and the AI assistant.

Seamless experience in the programming environment: Users can interact directly with the RAG system within their working environment using a VSCode Extension. This is convenient because:

- **No context switching:** By eliminating the need for users to alternate between the code editor and web browser, this feature reduces distractions and boosts efficiency.
- **Direct integration into the project:** The RAG system can quickly access the source code being changed thanks to the VSCode plugin, offering solutions or recommendations without forcing users to copy and paste code into a different interface.

Supportive in IDE environment: The most often used IDE environment for web installers is Visual Studio Code, particularly when using more recent frameworks like Next.js. The RAG system’s VSCode integration enables:

- **Use code context:** From the Next.js structure folder (which includes pages, api, and app) to the file configuration (next.config.js, .env), the system may directly examine the project’s coding.
- **Faster debugging and code optimization:** The system may access the coding link to offer more precise answers than the standard web interface information when users need to debug, test logic, or optimize coding.

Security and privacy: When working with source code, especially in sensitive projects, users are often concerned about security issues. A VSCode Extension:

- **Local storage:** Allows the RAG system to operate locally on the user’s machine (when using an internal RAG model such as LLaMA), avoiding the need to upload source code to an external server.
- **Limit information leakage:** With extensions, source code and sensitive data do not need to be transferred to a third browser or server, reducing the risk of information disclosure.

4.4.3 Feedback loop

Even the most advanced models, such as GPT-4 [32] or local LLMs like LLaMA [48], are not immune to errors or biases. While they excel at generating high-quality responses, these models lack intrinsic mechanisms for discerning truth, consistency, or fairness without external guidance. To implement robust feedback loops in our system, we leverage a combination of automated and human-guided approaches, ensuring comprehensive oversight and continuous refinement.

- **Human-in-the-Loop (HITL) Feedback:** Users interacting with the VSCode extension provide explicit feedback on the relevance and accuracy of responses, particularly for sensitive or complex queries. For example, when a user submits a query with a code snippet, they can flag issues such as incorrect logic, incomplete explanations, or unhelpful suggestions. These inputs are used to refine the model’s understanding and response generation iteratively.
- **Reinforcement Learning from Human Feedback (RLHF):** RLHF is employed to align the model’s outputs with user preferences and ethical guidelines. By introducing a reward system based on user feedback, the model learns to prioritize accuracy, clarity, and helpfulness. This method ensures that the system adapts to the specific needs of the developer community.

Initial Implementation

For this implementation, we specifically prepared the data for version 15.1.0 of Next.js. In our initial efforts, we concentrated on establishing the foundational components of the system, which included designing the data structure, preparing and preprocessing the data, and configuring both the database and local Large Language Models (LLMs) to develop a functional prototype.

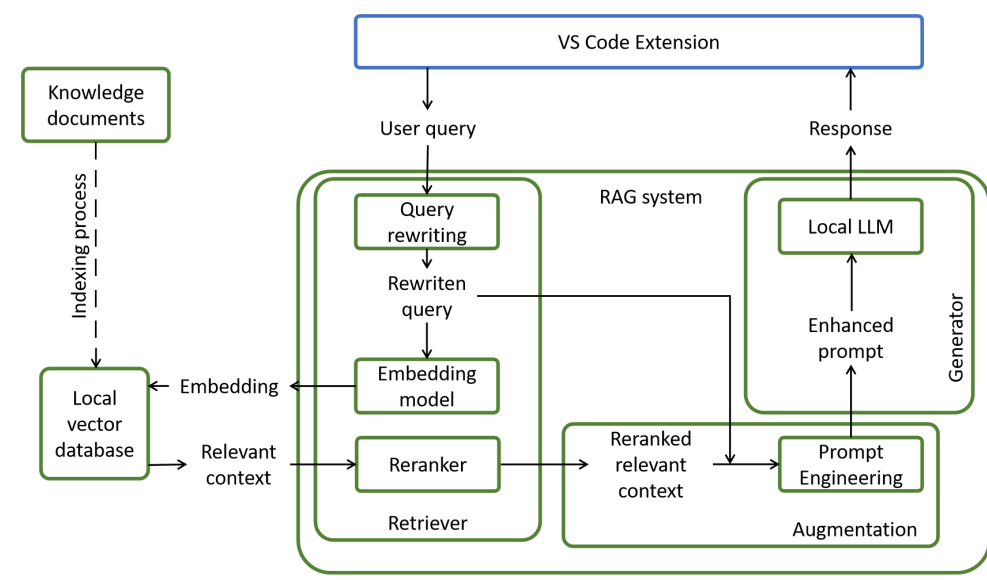


Figure 5.1: Our RAG Pipeline Design

5.1 Data Structure Design

Before delving into the structure and preparation of the data, it is important to discuss the data source. Naturally, the official documentation pages of the Next.js framework were our first consideration. Upon examination, the HTML files of the documentation were found to be well-formed and extractable. However, a critical limitation emerged: the official documentation does not maintain a comprehensive version history. The version selection section often changes its listed versions, making it unreliable for our intended use case.

Our goal is to provide users with the ability to select their current framework version and the version they wish to upgrade or downgrade to. This feature requires reliable and comprehensive documentation for all versions.

Fortunately, we discovered a more reliable data source: the official GitHub repository of the Next.js framework. This repository contains all versions of the framework's documentation in markdown format.

These markdown files include useful metadata, explicit sections, and well-organized code snippets, making them ideal for our use case.

5.1.1 Data structure design

The purpose of our data structure design is to store the Next.js documentation effectively. After analyzing the structure of the markdown files retrieved from the repository, the following key components were identified:

- **Metadata:** Contains information such as:
 - **title:** The title of the documentation page.
 - **nav_title:** The title displayed in the navigation bar.
 - **description:** A brief description of the page's content.
 - **related:** An optional field containing links to other related documentation pages.
- **Text:** The main content of the documentation, divided into sections and subsections using markdown syntax.
- **Code Snippets:** Includes:
 - **language:** Programming language used in the snippet (e.g., JavaScript or TypeScript).
 - **file:** Indicates the file or component to which the code belongs (Next.js follows a naming convention for files).
 - **switcher:** Indicates whether the code snippet has a counterpart in another language (e.g., JavaScript/TypeScript switcher).
 - **content:** The actual code content.
- **Version History:** A special section explicitly listing major updates to the documentation across versions. However, this section is present in only a small portion of the documentation.

Properties	Description	Note
name	Name of the field in the collection to create	Data type: String. Mandatory
dtype	Data type of the field	Mandatory
description	Description of the field	Data type: String. Optional
is_primary	Whether to set the field as the primary key field or not	Data type: Boolean (true or false). Mandatory for the primary key field
auto_id	Switch to enable or disable automatic ID (primary key) allocation	true or false . Mandatory for primary key field
max_length	Maximum byte length for strings allowed to be inserted.	[1, 65,535]. Mandatory for VARCHAR field. Multibyte characters may occupy more than one byte
dim	Dimension of the vector	Data type: Integer [1, 32,768]. Mandatory for dense vector field. Omit for sparse vector field
is_partition_key	Whether this field is a partition-key field	Data type: Boolean (true or false)

Table 5.1: Field schema properties(Manage Schema¹).

Based on these components, we designed a schema to organize and store the documentation data efficiently.

Collection: nextjs_docs

¹<https://milvus.io/docs/schema.md#Manage-Schema>

This single collection will store all data, including documents, sections, and code snippets. Each record will represent one of the following:

- A **document**: Represents a top-level page in the documentation.
- A **section**: Represents a subsection within a document.
- A **code snippet**: Represents a code example belonging to a section.

5.1.2 Fields in the Schema

Primary Scalar Fields

Field Name	Description
<code>entry_id</code> (Primary Key)	Unique identifier for each entry.
<code>tag</code>	A tag derived from <code>nav_title</code> to indicate the subject or topic of the entry. Useful for categorizing and filtering entries by subject.
<code>entry_type</code>	Type of entry (<code>document</code> , <code>section</code> , or <code>code_snippet</code>).
<code>parent_id</code>	The <code>entry_id</code> of the parent entry. For: • Sections: Links to the document or other section it belongs to. • Code snippets: Links to the section it belongs to.
<code>title</code>	Title of the document, section, or code snippet.
<code>description</code>	Description from <code>description</code> metadata.
<code>metadata</code>	JSON or string field for custom metadata (e.g., programming language, or related links).
<code>version</code>	Framework version associated with the entry (e.g., "15.1.0").

Table 5.2: Primary Scalar Fields in the Schema.

Text Fields

Field Name	Index	.
<code>text_content</code>	The raw text content (e.g., document or section body).	
<code>code_content</code>	The actual code snippet (only for entries of type <code>code_snippet</code>).	

Table 5.3: Caption

Embedding Vectors

Field Name	Index	Metric
<code>sparse_title_description</code>	SPARSE_INVERTED_INDEX	Inner Product
<code>dense_text_content</code>	HNSW	Cosine Similarity (Dimension: 1024)
<code>dense_code_snippet</code>	HNSW	Cosine Similarity (Dimension: 1024)

Table 5.4: Embedding Vectors in the Schema.

Partitioning Field: `version`

This field is used to partition entries by `version`. Partitioning aligns with Milvus's capability for managing partitions, ensuring efficient queries when users specify the framework version.

5.2 Data Preparation

As described in the previous section, our primary data source is the GitHub repository of the Next.js framework. Initially, we developed a crawler that recursively navigated and downloaded resources from the `docs` and `errors` folders of the repository. However, this approach faced significant limitations due to GitHub's API rate limits. After careful evaluation, we adopted a more efficient and native method to retrieve data directly from the GitHub repository.

Steps in Data Preparation:

- **Data Acquisition:** To extract the necessary files, we implemented a Python script that utilizes Git's sparse-checkout feature. This approach enabled us to selectively download specific folders, such as `docs` and `errors`, corresponding to a particular version tag. The script automates tasks including cloning the repository, configuring sparse-checkout, and checking out the desired version tag.
- **Data Organization:** The retrieved folders and their contents were stored in version-specific directories. For instance, for the version `v15.1.0`, the directory structure appeared as follows:

This structure provides an organized format, facilitating seamless access and retrieval of documentation resources for different framework versions.
- **Data Integrity Check:** To ensure the integrity and relevance of the dataset, the script included steps to validate and clean the retrieved data. Non-essential files and directories were removed, leaving only the targeted folders and their respective contents. This process ensured that the dataset was well-structured and ready for subsequent preprocessing.

5.3 Data Preprocessing

Once the data was acquired and organized, the next critical step was preprocessing it to ensure compatibility with the schema design for effective storage and retrieval. This process involved converting raw markdown files into structured formats through a preprocessing pipeline implemented in Python. The pipeline utilized markdown parsing libraries, embedding models, and structured data processing to extract meaningful information from the documentation.

Key Preprocessing Steps

- **Metadata Extraction:** The metadata fields, such as `title`, `description`, and `nav_title`, were extracted from markdown files using the `frontmatter` library. Additional metadata, such as `related` links, was included if available. Each document was assigned a unique `entry_id` to enable hierarchical referencing within the schema.

Example code for generating a unique entry ID:

```
1 def generate_entry_id(file_path):
2     relative_path = os.path.relpath(file_path)
3     entry_id = re.sub(r'[^a-zA-Z0-9]', '_', relative_path)
4     return entry_id.lower()
5
```

Listing 5.1: Generating Unique Entry IDs

- **Markdown Parsing:** Markdown files were parsed into components such as headings, paragraphs, lists, and code blocks using the `mistletoe` library. The root token, `Document`, represented the entire markdown file, while specific tokens such as `Heading`, `Paragraph`, and `List` enabled detailed content extraction while maintaining the document's hierarchy.
- **Content Extraction:** Each token's child elements were recursively processed to capture plain text, inline code, and hyperlinks. For example, `RawText` tokens represented basic text, `InlineCode` tokens identified inline code snippets, and `Link` tokens extracted hyperlink text and URLs.
- **Section Segmentation:** Logical sections were created based on `Heading` tokens in the markdown. Each section included a title, derived from the heading, and grouped content such as paragraphs, lists, or code blocks. This segmentation maintained the hierarchical structure necessary for efficient data retrieval.

- **Code Snippet Identification:** Code blocks were identified using `CodeFence` tokens and extracted along with relevant metadata, including the programming language, associated filename, and optional switcher flags. This information linked code snippets to their respective sections and allowed for language-specific queries.
- **Data Structuring:** The extracted sections were structured as dictionaries containing fields such as section title, textual content, and associated code snippets. This data organization was designed to align with the schema in the database for seamless integration.
- **Embedding Preparation:** Text and code content from each section were processed to prepare for embedding generation. Sparse embeddings were used for keyword indexing, while dense embeddings captured semantic meaning, enabling similarity-based queries.
- **Validation and Cleanup:** Parsed sections and their content were validated for completeness and correctness. Empty or incomplete sections were flagged for review, and non-essential formatting artifacts were removed to ensure clean and reliable data.

The `parse_markdown_sections` function, implemented using `mistletoe`, played a central role in these steps by iterating over the document tokens to extract structured information. For instance, a document with headings, paragraphs, and code blocks was transformed into hierarchical sections with associated metadata and content.

Example of parsing and organizing code snippets:

```

1  if isinstance(token, CodeFence):
2      code_info = {
3          "code": token.children[0].content if token.children else "",
4          "language": token.language or "text",
5          "filename": filename,
6          "switcher": switcher
7      }
8      current_section["code_snippets"].append(code_info)

```

Listing 5.2: Code Snippet Parsing

- **Embedding Generation:** Dense and sparse embeddings were generated for both text and code content using a pre-trained embedding model (`BGEM3EmbeddingFunction`). Dense vectors captured semantic meaning, while sparse vectors indexed key terms for efficient retrieval.
- **Batch Processing and Storage:** Processed entries were saved in batches as CSV files, with fields such as `entry_id`, `entry_type`, `text_content`, and embeddings. This modular strategy supported scalability and efficient handling of large datasets.

The preprocessing resulted in 4,714 data entries stored in a CSV file. A total of 481 files were processed, while 97 files were skipped because their content was generated by other files.

5.4 Database Setup

Setting up the database involved configuring Milvus for effective data storage, indexing, and retrieval. This process included deploying Milvus, defining the schema, creating indices, and inserting as well as retrieving data.

5.4.1 Deploying Milvus with Docker Compose

Milvus was deployed using Docker Compose. After installing Docker, the configuration file was downloaded, and the required containers were initialized. This setup resulted in three key containers:

- **milvus-etcd:** Responsible for managing metadata storage.
- **milvus-minio:** Manages object storage for the system.
- **milvus-standalone:** Handles vector indexing and retrieval services.

5.4.2 Installing PyMilvus

The PyMilvus library was installed to allow programmatic interaction with the Milvus server. To ensure compatibility, the version of PyMilvus matched the Milvus server version (2.4).

Schema Definition and Collection Creation

As outlined in the data structure design, the database schema was implemented with the collection name `nextjs_docs`.

5.4.3 Creating Indices

To optimize retrieval performance, indices were created for each of the three vector fields:

- **Sparse Index:** Built for the `sparse_title_description` field using an inverted index with inner product (IP) as the metric type.
- **Dense Text Index:** Applied to the `dense_text_content` field using the HNSW algorithm with cosine similarity.
- **Dense Code Index:** Created for the `dense_code_snippet` field, also utilizing the HNSW algorithm with cosine similarity.

5.4.4 Inserting Data into the Collection

The preprocessed data, stored as a CSV file, was read and validated to ensure all data types matched the schema requirements.

5.4.5 Hybrid search implementation

The hybrid search function was implemented as a prototype to illustrate the retrieval process by integrating sparse and dense embeddings.

Workflow of Hybrid Search:

1. **Input and Initialization:** Sparse and dense embeddings are provided as inputs, along with weights to balance their influence. Sparse embeddings are processed using the `sparse_search` function, while dense embeddings are handled by the `dense_text_search` function.
2. **Sparse Search:** This step retrieves entries based on keyword relevance. The output includes the entry's metadata and a distance metric.
3. **Dense Search:** This step retrieves entries based on semantic similarity, outputting the entry's metadata and a distance metric.
4. **Combining Results:** The results from sparse and dense searches are merged using `entry_id` as the common key. A combined score is calculated for each entry:

$$\text{Combined Score} = (\text{Sparse Distance} \times \text{Sparse Weight}) + (\text{Dense Distance} \times \text{Dense Weight})$$

If an entry is missing in either result, its corresponding distance is set to zero.

5. **Sorting and Limiting Results:** The entries are ranked by their combined scores in descending order. The top `limit` results are returned, demonstrating the hybrid retrieval process.

5.5 Local LLMs setup

In the development of RAG systems, the choice of a robust and secure LLM is critical for achieving high-quality responses and ensuring data privacy. To support our RAG pipeline, we installed and configured Ollama², a platform specifically designed to facilitate the deployment of pre-trained language models on local infrastructure. This section describes the setup and integration of Ollama into our system.

²Ollama - <https://ollama.com/>

5.5.1 Setup Ollama

The setup process begins by installing the Ollama application, which is compatible with various operating systems including Windows, macOS, and Linux. Once installed, Ollama provides a straightforward interface to download and configure pre-trained models without requiring fine-tuning.

The installation of specific models is conducted via the command prompt or terminal, with available models conveniently listed on <https://ollama.com/search>. The command to install a model is `ollama pull <model_name>`.

Once a model is installed, it is stored locally, ensuring a high level of data privacy and security — an essential feature for applications dealing with sensitive or proprietary information. To test the model, we can use the command `ollama run <model_name>`.

Furthermore, the platform's lightweight design enhances compatibility across a variety of hardware configurations, making it particularly suitable for environments with limited computational resources. This flexibility makes Ollama an optimal choice for deploying local LLMs in diverse use cases.

5.5.2 Integration with the RAG System

To integrate Ollama into the RAG workflow, we use the Ollama API for direct interaction with the locally installed models. The endpoint for the API is:

POST `http://localhost:11434/api/generate`

The POST request body should be in JSON format, specifying the desired model and the prompt to be processed. An example request body is shown below:

```
{
  "model": "qwen",
  "prompt": "What is RAG?"
}
```

The API provides a straightforward and efficient method for passing queries and receiving responses from the model.

Alternatively, Ollama models can also be integrated using the Python library LangChain³. With LangChain, the locally installed Ollama models can be accessed and utilized as part of a more comprehensive pipeline, allowing for advanced customization and ease of interaction. This flexibility enables developers to build robust RAG systems that leverage the full potential of Ollama's capabilities while adapting to diverse use cases.

³LangChain - <https://python.langchain.com/docs/introduction/>

6.1 Evaluation metrics

6.1.1 Retriever metrics:

Recall@K: Recall@K measures the proportion of relevant documents retrieved within the top K results for a query.

$$\text{Recall@K} = \frac{|\text{Relevant Documents} \cap \text{Retrieved Top K Documents}|}{|\text{Relevant Documents}|}$$

where:

- $|\text{Relevant Documents}|$ is the total number of relevant documents for the query.
- $|\text{Relevant Documents} \cap \text{Retrieved Top K Documents}|$ is the number of relevant documents retrieved in the top K results.

Precision@K: Precision@K evaluates the accuracy of the retrieved documents within the top K results.

$$\text{Precision@K} = \frac{|\text{Relevant Documents} \cap \text{Retrieved Top K Documents}|}{|\text{Retrieved Top K Documents}|}$$

where:

- $|\text{Retrieved Top K Documents}|$ is the total number of documents retrieved in the top K results.

Latency: Latency measures the time taken to retrieve documents for a query.

$$\text{Latency (ms)} = \text{Time when results are returned} - \text{Time when query is submitted}$$

Coverage: Coverage assesses the percentage of queries for which at least one relevant document is retrieved.

$$\text{Coverage} = \frac{\text{Number of Queries with Relevant Results Retrieved}}{\text{Total Number of Queries}} \times 100$$

6.1.2 Generator metrics:

ROUGE score: ROUGE evaluates the overlap between the generated text and reference outputs. For example, ROUGE-N is defined as:

$$\text{ROUGE-N} = \frac{\sum_{\text{ref} \in \text{References}} \sum_{\text{ngram} \in \text{Generated}} \text{Count}(\text{ngram})}{\sum_{\text{ref} \in \text{References}} \sum_{\text{ngram} \in \text{ref}} \text{Count}(\text{ngram})}$$

Relevance: Relevance is assessed manually by rating how well the generated outputs align with the intent of user queries. Scores are averaged across all queries:

$$\text{Relevance} = \frac{\sum_{i=1}^N \text{Relevance Score for Query } i}{N}$$

Factual accuracy: Factual accuracy measures the percentage of generated outputs that are factually correct based on the retrieved data:

$$\text{Factual accuracy} = \frac{\text{Number of Factually Correct Outputs}}{\text{Total Number of Outputs}} \times 100$$

Latency: Latency for the generator is calculated similarly to the retriever:

$$\text{Latency (ms)} = \text{Time when output is generated} - \text{Time when retrieval is completed}$$

6.1.3 System metrics:

Usability: Usability is evaluated through user feedback, typically using a Likert scale (e.g., 1–5). The overall usability score is averaged:

$$\text{Usability} = \frac{\sum_{i=1}^N \text{User Rating for Task } i}{N}$$

Task success rate: Task success rate measures the percentage of tasks users complete successfully:

$$\text{Task success rate} = \frac{\text{Number of Successfully Completed Tasks}}{\text{Total Number of Tasks}} \times 100$$

Integration latency: Integration latency measures the total response time of the system, from query input to output generation:

$$\text{Integration latency (ms)} = \text{Time when output is delivered} - \text{Time when query is submitted}$$

6.2 Evaluation methodology

6.2.1 Retriever evaluation:

We will use the pre-defined dataset with known relevant results to benchmark the retriever’s performance across query types:

- **Simple queries:** Basic queries requiring straightforward retrieval of documents.
- **Complex queries:** Multi-faceted queries involving multiple conditions or ambiguities.
- **Context-dependent queries:** Queries that rely on contextual understanding of the current repository.

6.2.2 Generator evaluation:

Provide the generator with retriever outputs and evaluate the quality of generated content using:

- **Automatic metrics:** ROUGE scores to quantify similarity and overlap with reference outputs.
- **Human evaluation:** Involve domain experts to rate the relevance, accuracy, and quality of generated outputs on a predefined scale.

Include edge cases in testing, such as incomplete or ambiguous retriever outputs, to ensure the generator’s robustness.

6.2.3 System evaluation:

Conduct controlled user testing sessions where participants use the VSCode extension to complete specific coding tasks.

Gather qualitative feedback on the system’s usability and design through post-session interviews or open-ended survey questions.

Future plan

7.1 Overview

As of December 2024, the research tasks are nearing completion, and the transition to implementation is beginning.

7.2 Current tasks: late December 2024

Early development efforts will begin to validate the feasibility of the proposed approach. This includes the data curation, chunking, and setting up local LLM.

7.3 Upcoming tasks: January 2025 and Beyond

1. **Collecting and organizing the dataset (January 2025):** Data collection and preprocessing will begin in January, focusing on ensuring dataset quality and relevance.
2. **Preparing local LLM and embedding system (January 2025):** The local LLM and embedding system will be configured and tested to handle efficient information retrieval.
3. **Configuring the vector database (January 2025):** A vector database will be set up to store and retrieve embeddings efficiently, enabling core retrieval functionalities.
4. **Implementing query handling and prompt management (February 2025):** Mechanisms for processing user queries and managing prompts will be developed to ensure smooth user interactions.
5. **Evaluating retriever and generator (February 2025):** The retrieval and generation components will be assessed for accuracy, efficiency, and performance.
6. **Integrating into VSCode (February 2025):** The system will be integrated as an extension into VSCode to enhance developer usability and accessibility.
7. **Implementing user interface for interactions (March 2025):** A user-friendly interface will be developed to facilitate seamless interaction between users and the system.
8. **Evaluating the extension's performance (March 2025):** Comprehensive performance testing of the VSCode extension will be conducted to ensure reliability and usability.
9. **Optimizing the system (March 2025):** System optimization efforts will focus on improving speed, accuracy, and overall user experience.
10. **Deployment (April 2025):** The fully developed and optimized system will be deployed for production use.
11. **Writing final report (April 2025):** A comprehensive report summarizing the research, development, evaluation, and findings will be prepared and submitted.

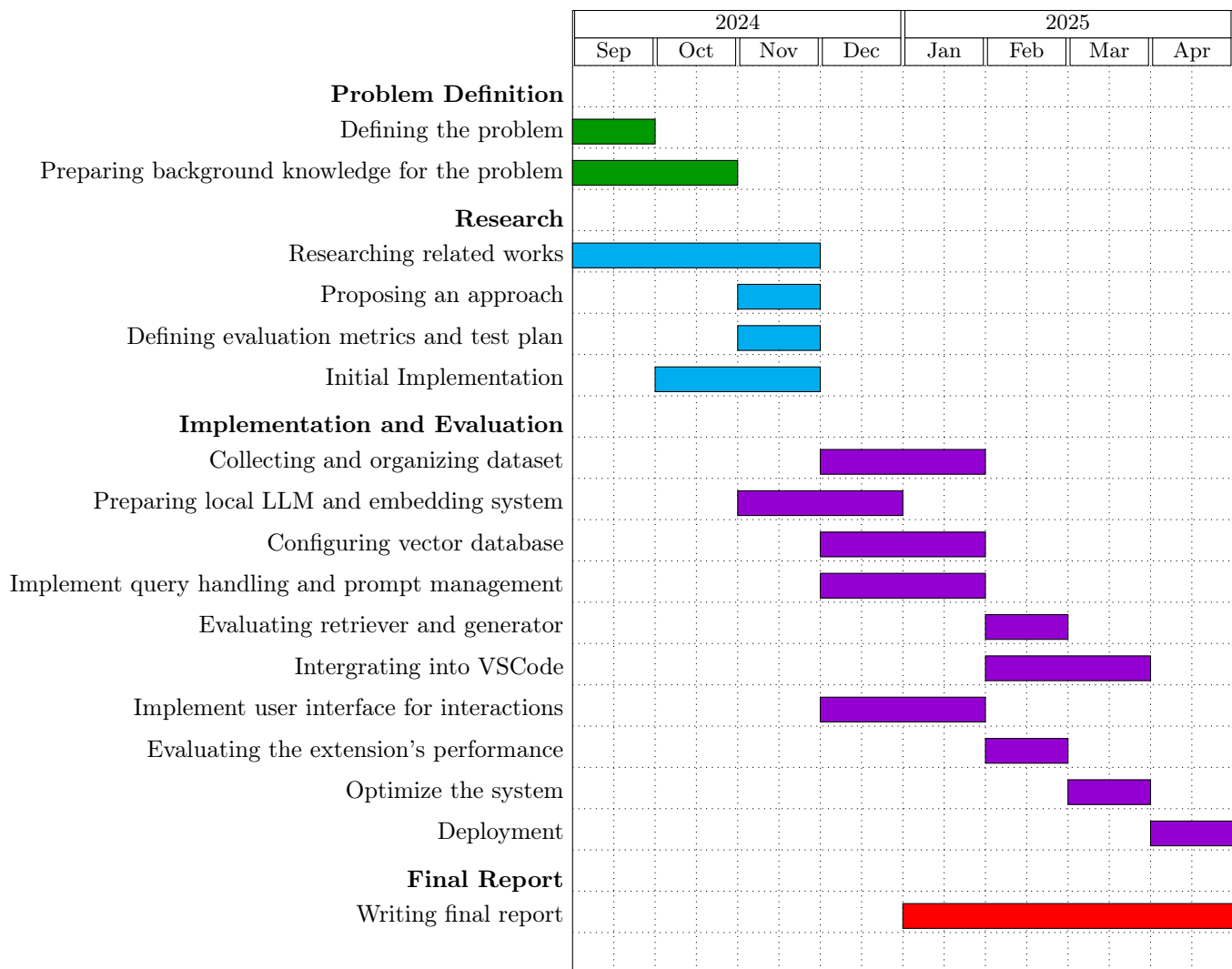


Figure 7.1: Future plan

Bibliography

- [1] A. Asai et al. “SELF-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection”. In: *arXiv preprint* arXiv:2310.11511v1 (2023). Available at <https://arxiv.org/abs/2310.11511>.
- [2] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. “Layer Normalization”. In: *stat* 1050 (2016), p. 21.
- [3] Jinze Bai et al. “Qwen Technical Report”. In: *arXiv preprint* arXiv:2309.16609 (2023). Available at <https://arxiv.org/abs/2309.16609>.
- [4] Erik Bernhardsson. *Approximate Nearest Neighbors Oh Yeah*. Online. Available at <https://github.com/spotify/annoy>.
- [5] D. Brown. *Rank-BM25: A Collection of BM25 Algorithms in Python*. Online. Available at <https://doi.org/10.5281/zenodo.4520057>. 2020.
- [6] Tom B. Brown et al. *Language Models are Few-Shot Learners*. 2020. arXiv: 2005.14165 [cs.CL]. URL: <https://arxiv.org/abs/2005.14165>.
- [7] Marcel Bruch, Martin Monperrus, and Mira Mezini. “Learning from Examples to Improve Code Completion Systems”. In: *Proceedings of the 7th Joint Meeting of the European Software Engineering Conference and ACM Symposium on the Foundations of Software Engineering (ESEC/FSE)*. Amsterdam, Netherlands, 2009.
- [8] Jianlv Chen et al. *BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. 2024. arXiv: 2402.03216 [cs.CL]. URL: <https://arxiv.org/abs/2402.03216>.
- [9] Mark Chen et al. *Evaluating Large Language Models Trained on Code*. 2021. arXiv: 2107.03374 [cs.LG]. URL: <https://arxiv.org/abs/2107.03374>.
- [10] Qi Chen et al. *SPTAG: A Library for Fast Approximate Nearest Neighbor Search*. Available at <https://github.com/Microsoft/SPTAG>. 2018.
- [11] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. Available at <https://arxiv.org/abs/1810.04805>. 2019. arXiv: 1810.04805 [cs.CL]. URL: <https://arxiv.org/abs/1810.04805>.
- [12] Matthijs Douze et al. “The Faiss library”. In: (2024). arXiv: 2401.08281 [cs.LG].
- [13] *Elasticsearch*. Online. Available at <https://github.com/elastic/elasticsearch>.
- [14] Yunfan Gao et al. “Retrieval-Augmented Generation for Large Language Models: A Survey”. In: *arXiv preprint* arXiv:2312.10997 (2023). Available at <https://arxiv.org/abs/2312.10997>.
- [15] Kaiming He et al. “Deep residual learning for image recognition”. In: *Proceedings of the IEEE conference on computer vision and pattern recognition*. 2016, pp. 770–778.
- [16] Pengfei He et al. “Exploring Demonstration Retrievers in RAG for Coding Tasks: Yeas and Nays!”. In: *arXiv preprint* arXiv:2410.09662v1 (2024). Available at <https://arxiv.org/abs/2410.09662>.
- [17] Sepp Hochreiter and Jürgen Schmidhuber. “Long Short-term Memory”. In: *Neural computation* 9 (Dec. 1997), pp. 1735–80. DOI: 10.1162/neco.1997.9.8.1735.
- [18] Marko Hostnik and Marko Robnik-Sikonja. “Retrieval-Augmented Code Completion for Local Projects Using Large Language Models”. In: *arXiv preprint* arXiv:2408.05026v1 (2024). Available at <https://arxiv.org/abs/2408.05026>.

- [19] Binyuan Hui et al. “Qwen2.5-Coder Technical Report”. In: *arXiv preprint* arXiv:2409.12186 (2024). Available at <https://arxiv.org/abs/2409.12186>.
- [20] Gautier Izacard et al. “Unsupervised Dense Information Retrieval with Contrastive Learning”. In: *Transactions on Machine Learning Research* (2022). ISSN: 2835-8856. URL: <https://openreview.net/forum?id=jKN1pXi7b0>.
- [21] O. Jafari et al. “A Survey on Locality Sensitive Hashing Algorithms and Their Applications”. In: *arXiv preprint* arXiv:2102.08942 (2021). Available at <https://arxiv.org/abs/2102.08942>.
- [22] Jeff Johnson, Matthijs Douze, and Hervé Jégou. “Billion-scale Similarity Search with GPUs”. In: *arXiv preprint* arXiv:1702.08734 (2017). Available at <https://arxiv.org/abs/1702.08734>.
- [23] Aadit Kshirsagar. “Enhancing RAG Performance Through Chunking and Text Splitting Techniques”. In: *International Journal of Scientific Research in Computer Science, Engineering and Information Technology* 10 (Sept. 2024), pp. 151–158. DOI: 10.32628/CSEIT2410593.
- [24] Mandar Kulkarni et al. “Reinforcement Learning for Optimizing RAG for Domain Chatbots”. In: *Proceedings of the AAAI 2024 Workshop on Synergy of Reinforcement Learning and Large Language Models*. 2024.
- [25] Patrick Lewis et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In: *arXiv preprint* arXiv:2005.11401 (2020).
- [26] Jie Li et al. “The Design and Implementation of a Real Time Visual Search System on JD E-commerce Platform”. In: *arXiv preprint* arXiv:1908.07389 (2019). Available at <https://arxiv.org/abs/1908.07389>.
- [27] Jie Li et al. “The Design and Implementation of a Real Time Visual Search System on JD E-commerce Platform”. In: *arXiv preprint* arXiv:1908.07389 (2019). Available at <https://arxiv.org/abs/1908.07389>.
- [28] Xinze Li et al. “Structure-Aware Language Model Pretraining Improves Dense Retrieval on Structured Data”. In: *arXiv preprint* arXiv:2305.19912 (2023). Available at <https://arxiv.org/abs/2305.19912>.
- [29] Y. A. Malkov and D. A. Yashunin. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2018), pp. 824–836. DOI: 10.1109/TPAMI.2018.2889473.
- [30] Tomas Mikolov et al. “Efficient Estimation of Word Representations in Vector Space”. In: *arXiv preprint arXiv:1301.3781* (2013).
- [31] Bhaskar Mitra and Nick Craswell. “Neural Models for Information Retrieval”. In: *arXiv preprint* arXiv:1705.01509v1 (May 2017). Available at <https://arxiv.org/abs/1705.01509>.
- [32] OpenAI et al. *GPT-4 Technical Report*. 2024. arXiv: 2303.08774 [cs.CL]. URL: <https://arxiv.org/abs/2303.08774>.
- [33] *PASE: PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension*. Online. Available at <https://www.researchgate.net/publication/341744935>.
- [34] Alec Radford and Karthik Narasimhan. “Improving Language Understanding by Generative Pre-Training”. In: 2018. URL: <https://api.semanticscholar.org/CorpusID:49313245>.
- [35] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. Tech. rep. Available at https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf. OpenAI, 2019.
- [36] Colin Raffel et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. Available at 1910.10683. 2023. arXiv: 1910.10683 [cs.LG]. URL: <https://arxiv.org/abs/1910.10683>.
- [37] O. Ram et al. “In-Context Retrieval-Augmented Language Models”. In: *arXiv preprint* arXiv:2302.00083v3 (2023). Available at <https://arxiv.org/abs/2302.00083>.
- [38] Nils Reimers and Iryna Gurevych. “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Nov. 2019. URL: <https://arxiv.org/abs/1908.10084>.
- [39] Stephen E. Robertson and Hugo Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond”. In: *Found. Trends Inf. Retr.* 3 (2009), pp. 333–389. URL: <https://api.semanticscholar.org/CorpusID:207178704>.
- [40] Baptiste Rozière et al. “Code Llama: Open Foundation Models for Code”. In: *arXiv preprint* arXiv:2308.12950 (2023). Available at <https://arxiv.org/abs/2308.12950>.

- [41] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. “Learning representations by back-propagating errors”. In: *Nature* 323 (1986). Available at <https://doi.org/10.1038/323533a0>, pp. 533–536.
- [42] Saksham Sachdev et al. “Retrieval on Source Code: A Neural Code Search”. In: *Proceedings of the 2nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL)*. Philadelphia, PA, USA, 2018.
- [43] Pranab Sahoo et al. *A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications*. 2024. arXiv: 2402.07927 [cs.AI]. URL: <https://arxiv.org/abs/2402.07927>.
- [44] Alexey Svyatkovskiy et al. “IntelliCode Compose: Code Generation using Transformer and Retrieval-based Models”. In: *Proceedings of the ACM/IEEE International Conference on Software Engineering (ICSE)*. 2020.
- [45] Hanzhuo Tan et al. “Prompt-based Code Completion via Multi-Retrieval Augmented Generation”. In: *arXiv preprint arXiv:2405.07530v1* (2024). Available at <https://arxiv.org/abs/2405.07530>.
- [46] Gemini Team et al. *Gemini: A Family of Highly Capable Multimodal Models*. 2024. arXiv: 2312.11805 [cs.CL]. URL: <https://arxiv.org/abs/2312.11805>.
- [47] Hugo Touvron et al. *Llama 2: Open Foundation and Fine-Tuned Chat Models*. 2023. arXiv: 2307.09288 [cs.CL]. URL: <https://arxiv.org/abs/2307.09288>.
- [48] Hugo Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*. 2023. arXiv: 2302.13971 [cs.CL]. URL: <https://arxiv.org/abs/2302.13971>.
- [49] Ashish Vaswani et al. “Attention Is All You Need”. In: *arXiv preprint arXiv:1706.03762* (2017). Available at <https://arxiv.org/pdf/1706.03762>.
- [50] Jianguo Wang et al. “Milvus: A Purpose-Built Vector Data Management System”. In: *Proceedings of the 2021 International Conference on Management of Data (SIGMOD ’21)*. Virtual Event, China, 2021, pp. 2624–2638. DOI: 10.1145/3448016.3457550.
- [51] Liang Wang et al. *Improving Text Embeddings with Large Language Models*. 2024. arXiv: 2401.00368 [cs.CL]. URL: <https://arxiv.org/abs/2401.00368>.
- [52] Liang Wang et al. *Text Embeddings by Weakly-Supervised Contrastive Pre-training*. 2024. arXiv: 2212.03533 [cs.CL]. URL: <https://arxiv.org/abs/2212.03533>.
- [53] X. Wang et al. “KnowledGPT: Enhancing Large Language Models with Retrieval and Storage Access on Knowledge Bases”. In: *arXiv preprint arXiv:2308.11761v1* (2023). Available at <https://arxiv.org/abs/2308.11761>.
- [54] Chuangxian Wei et al. “AnalyticDB-V: A Hybrid Analytical Engine Towards Query Fusion for Structured and Unstructured Data”. In: *Proceedings of the VLDB Endowment* 13.12 (2020), pp. 3152–3165. DOI: 10.14778/3415478.3415541.
- [55] Jason Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. 2023. arXiv: 2201.11903 [cs.CL]. URL: <https://arxiv.org/abs/2201.11903>.
- [56] Di Wu et al. “Repoformer: Selective Retrieval for Repository-Level Code Completion”. In: *Proceedings of the 41st International Conference on Machine Learning*. Vol. 235. Vienna, Austria: PMLR, 2024.
- [57] Aiyuan Yang et al. *Baichuan 2: Open Large-scale Language Models*. 2023. arXiv: 2309.10305 [cs.CL]. URL: <https://arxiv.org/abs/2309.10305>.
- [58] Chanwoong Yoon et al. “COMPACT: Compressing Retrieved Documents Actively for Question Answering”. In: *Korea University and AIGEN Sciences* (2024). Authors: Chanwoong Yoon, Taewhoo Lee, Hyeon Hwang, Minbyul Jeong, and Jaewoo Kang.
- [59] Fengji Zhang et al. “RepoCoder: Repository-Level Code Completion Through Iterative Retrieval and Generation”. In: *arXiv preprint arXiv:2303.12570* (2023). Available at <https://arxiv.org/abs/2303.12570>.
- [60] Xinyu Zhang et al. “MIRACL: A Multilingual Retrieval Dataset Covering 18 Diverse Languages”. In: *Transactions of the Association for Computational Linguistics* 11 (Sept. 2023), pp. 1114–1131. ISSN: 2307-387X. DOI: 10.1162/tac1_a_00595. eprint: https://direct.mit.edu/tac1/article-pdf/doi/10.1162/tac1_a_00595/2157340/tac1_a_00595.pdf. URL: https://doi.org/10.1162/tac1%5C_a%5C_00595.
- [61] Xinyu Zhang et al. “Toward Best Practices for Training Multilingual Dense Retrieval Models”. In: *ACM Trans. Inf. Syst.* 42.2 (Sept. 2023). ISSN: 1046-8188. DOI: 10.1145/3613447. URL: <https://doi.org/10.1145/3613447>.

- [62] Ziyao Zhang et al. “LLM Hallucinations in Practical Code Generation: Phenomena, Mechanism, and Mitigation”. In: *arXiv preprint* arXiv:2409.20550v1 (2024). Available at <https://arxiv.org/abs/2409.20550>.